AuxMark: Tracing Against Unauthorized Distillation via Behavioral Watermarking

Submission ID: 1737

***Abstract*—In unauthorized model distillation, distillers use outputs from proprietary models to train student models without the providers' permission. A practical anti-distillation mechanism must reliably trace distillation and provide strong evidence that a suspect model was trained on the provider's outputs, while adding little operational overhead. We systematize existing anti-distillation techniques and show that they only partially meet these requirements. We provide new empirical evidence of these limitations in several prominent tracing methods. These findings motivate a setting in which providers coordinate the anti-distillation mechanism across the full model-serving pipeline. In this setting, we introduce `AuxMark`, a behavioral watermarking method for tracing unauthorized distillation of language model agents. `AuxMark` inserts safe auxiliary tool calls into teacher trajectories to embed behaviors that students can learn during distillation. Traffic screening then identifies suspicious trajectories for further investigation. The provider constructs targeted probes from these trajectories to test whether a suspect student has learned the embedded behaviors. In our evaluation covering three agent benchmarks and eight teacher-student model pairs, `AuxMark` detects all 24 distilled students with no false positives in 48 evaluations of clean models. It preserves task utility while remaining effective under the evaluated common procedures, demonstrating its practicality in real-world deployments.**

1. Introduction

Model distillation transfers capabilities from a teacher model to a student model through teacher-generated supervision [1]. In language models and autonomous agents, fine-tuning on teacher reasoning and tool-use trajectories provides an effective way to acquire complex behaviors at low computational cost [2–5]. However, this efficiency creates strong economic incentives to harvest proprietary outputs without authorization. By systematically querying commercial model APIs, competitors can acquire state-of-the-art capabilities while bypassing substantial research and training investments [6]. Recent disclosures from leading AI providers indicate that unauthorized distillation campaigns now operate at an industrial scale [6–8], presenting acute intellectual property risks across the AI ecosystem.

Provider investigations reveal that distillation campaigns rely on organized collection pipelines designed to evade front-end defenses. Anthropic documented campaigns across multiple laboratories involving more than 16 million exchanges with Claude, using coordinated accounts and proxy networks to harvest reasoning, coding, and tool-use trajectories [6]. Google similarly observed systematic extraction attempts targeting proprietary capabilities, including reasoning traces [8]. OpenAI reported the use of third-party routing infrastructure and extensive data cleaning prior to student fine-tuning [7]. These reports demonstrate that unauthorized extraction is coordinated, adaptive, and paired with aggressive data sanitization. Crucially, they also expose a fundamental verification gap: detecting suspicious request volumes or coordinated accounts demonstrates data collection, but does not prove whether a deployed student model was actually trained on those outputs.

Existing defenses address different stages of distillation and supply different forms of evidence. At the service boundary, traffic screening inspects request volumes and account patterns to detect automated collection [9–11], while access controls restrict endpoints or suppress unsummarized reasoning chains [7]. At generation time, output-side defenses perturb token sampling or rewrite intermediate reasoning to degrade training utility or leave statistical artifacts [12–14]. Proactive watermarking embeds secret signals in teacher outputs to audit whether student models inherit them [15, 16]. Alternatively, passive fingerprinting inspects naturally occurring similarities in output style or tool-use execution graphs without modifying teacher outputs [17–19]. Each route provides useful operational intelligence, but its underlying assumptions dictate what a provider can conclude from a traffic alert, an embedded signal, or a behavioral match.

A practical defense must reliably detect distillation and provide evidence specifically tied to the provider's outputs. It must operate under black-box student access, introduce minimal overhead to preserve the teacher's task utility, and remain effective when the distiller cleans, rewrites, truncates, or dilutes the collected trajectories. Existing approaches only partially satisfy these requirements: traffic screening leaves the training-use question open, passive fingerprinting lacks attribution specificity, and existing watermarks either disrupt structured tool calling or degrade under data processing. We therefore ask:

> *How can a provider obtain reliable evidence of distillation under black-box student access when the student's training data are unknown and subject to adversarial data processing?*

1.1. Our Contributions

Our contributions are twofold.

Systematization and new empirical evidence. Our first contribution is a systematic evaluation of existing anti-distillation techniques across their access requirements, operational costs, evidential strength, and resilience to data processing (Table 11). We complement this taxonomy with new analytical and empirical evidence exposing fundamental ambiguities in prominent detection methods. For reasoning-prefill detection [19], controlled comparisons on public problems show that shared conventional derivations (such as standard differential-equation integrating factors) and alternative prompt prefixes can produce substantial answer overlap without teacher distillation. For AgentEcho [17], we construct an analytical counterexample showing that an 11-node graph with a single edge and an 11-node complete 10-edge chain yield proportional feature vectors and an identical maximal dependency similarity ($S_{\texttt{dep}} = 1.0$). Furthermore, our trajectory evaluations show that reasoning distillation concurrently increases behavioral similarity toward multiple unrelated models. These findings demonstrate that unwatermarked behavioral similarity cannot uniquely identify a claimed training source.

Detection through retained behavioral evidence. Our second contribution connects interventions during model serving with post-hoc verification of a suspect student. In our setting, the provider creates and privately records behavioral evidence while serving trajectories. Service-side traffic screening subsequently identifies an anomalous candidate set of suspicious interactions $\mathcal{S}$ for auditing. Crucially, the provider does not need to know which specific interactions entered the distiller's training set $\mathcal{D}$, nor assume that $\mathcal{D} \subseteq \mathcal{S}$. Instead, our main insight is to audit the suspect student using targeted behavioral probes derived from the retained evidence, relying on the statistical signal surviving in the non-empty intersection $\mathcal{S} \cap \mathcal{D}$ (Figure 1).

Embedding learnable auxiliary behaviors. We instantiate this approach in `AuxMark`, a behavioral watermarking method for language model agents (Figure 2). Rather than altering text tokens or modifying core task actions, `AuxMark` inserts safe auxiliary tool calls into teacher trajectories immediately following normal core actions. Auxiliary calls are restricted to a pre-classified safe tool subset $\mathcal{F}_{\texttt{safe}}$ and strictly bound to parameter values drawn from the interaction context, ensuring that they execute safely and appear natural. A keyed pseudo-random schedule and candidate scoring policy favor repeatable core–auxiliary action pairs, instilling consistent behavioral habits that the student learns during distillation. Sparse insertion limits additional interaction steps while preserving the teacher's core task execution and accuracy. Each successful insertion creates a private evidence card recording the execution context, the core action, and the injected auxiliary parameters.

Paired-probe verification and trace attribution. To audit a suspect student, observing an auxiliary call in isolation is insufficient because an untrained model might occasionally invoke the tool naturally. `AuxMark` resolves this ambiguity through paired probes constructed from the retained evidence cards. For each card, the provider creates a real probe preserving the recorded core action and a counterfactual control probe that replaces the core action with a semantically equivalent alternative achieving the same goal. The provider queries the black-box student and tests whether it preferentially reproduces the target auxiliary behavior in real contexts over control contexts. Aggregating probe variants at the card level eliminates intra-card dependence, and the detection rule requires both a one-sided exact sign test ($p < 0.05$) and a minimum full-hit margin (≥ 0.05). The same evidence cards further support trace-level attribution, ranking candidate trajectories to identify specific samples included in the adversary's training set.

Empirical evaluation. We evaluate `AuxMark` across three agent benchmarks (BFCL, SWE-bench, and Telecom) and eight teacher–student pairs spanning four student model families. In an audit setting where the provider inspects 100 suspicious trajectories of which only 50 were used for distillation, `AuxMark` detects all 24 distilled student models ($p < 10^{-3}$ to 10^{-14}) while producing zero false positives across 48 clean-model evaluations (all $p \geq 0.062$). Watermark embedding preserves teacher utility: task accuracy changes by at most 3.0 percentage points, with mean interaction length increasing by only 0.60 steps. Furthermore, `AuxMark` remains effective under aggressive data processing, successfully detecting students after 1:10 dilution with clean data, mixing across three independent teachers, semantic rewriting, trajectory truncation, and adaptive LLM sanitization explicitly targeting auxiliary behaviors.

2. Problem Formulation and Preliminaries

2.1. Model Distillation

LLM interacts with an external environment to solve a user task through multiple rounds of reasoning and tool use [20, 21]. An model trajectory records intermediate reasoning and tool-use behaviors. Given a user query q, an LLM $\mathcal{M}$ interacts with the environment over T steps. At each step i, based on the query q and the interaction history h_i, the agent generates a thought t_i and an action $a_i = (f_i, p_i)$, where f_i denotes the selected tool and p_i denotes its parameters. The environment executes the action and returns an observation o_i. A complete interaction trajectory τ is formally represented as:

$$\tau = \{(t_i, a_i, o_i)\}_{i=1}^{T}, \text{ where } (t_i, a_i) = \mathcal{M}(q, h_i). \quad (1)$$

Model distillation [1, 4, 5, 22] aims to transfer such interaction behaviors from a powerful teacher model $\mathcal{M}_t$ to a student model $\mathcal{M}_s$. The teacher first generates a set of N trajectories $(q_j, \tau_j)_{j=1}^{N}$. In model distillation, we consider hard distillation based on supervised fine-tuning [23]. Through this process, the student can learn the teacher's reasoning and tool-use behaviors from its interaction trajectories. The student model is trained on $(q_j, \tau_j)_{j=1}^{N}$, where previous thoughts, actions, and observations are used as context, while the teacher's thoughts and actions are used as supervision [5, 22].

2.2. Model Watermarking

To protect the intellectual property of a teacher agent $\mathcal{M}_t$, the model owner employs a secret key k to embed watermark signals in real time during the generation of trajectories τ. After attackers query the model to obtain a watermarked dataset $\mathcal{D}$, they attempts to clone the teacher's capabilities by training an base model $\mathcal{M}_c$ on $\mathcal{D}$ via a distillation algorithm to get the student model $\mathcal{M}_s$. Then model owner audits suspicious student models using a private probe set $\mathcal{P}_k$. A practical anti-distillation watermark should satisfy the following three requirements:

Effectiveness. An effective watermark must reliably transfer to the student model during distillation while maintaining a minimal false positive rate on independent models. During verification, the owner queries the suspicious model using the probe set $\mathcal{P}_k$ and computes a p-value to evaluate the watermark signal. A distilled student model $\mathcal{M}_s$ should exhibit statistically significant evidence of the watermark, satisfying $p\text{-}\texttt{value} < p$, whereas a clean model $\mathcal{M}_c$ should yield $p\text{-}\texttt{value} > p$.

Harmlessness. The real-time watermarking mechanism should not degrade the normal operation of the teacher model $\mathcal{M}_t$. This entails two main aspects: task utility and inference efficiency. Specifically, the watermarked teacher model must maintain a success rate on the same tasks comparable to its unwatermarked counterpart, and the watermark embedding process should not significantly inflate the length or computational cost of the generated trajectories.

Robustness. The watermark must resist data modifications applied by attackers. Even if the attacker alters the dataset through data flooding, semantic rewriting, truncation, and adaptive cleaning attacks. the watermark signal should remain detectable in the student model.

2.3. Access and Threat Model

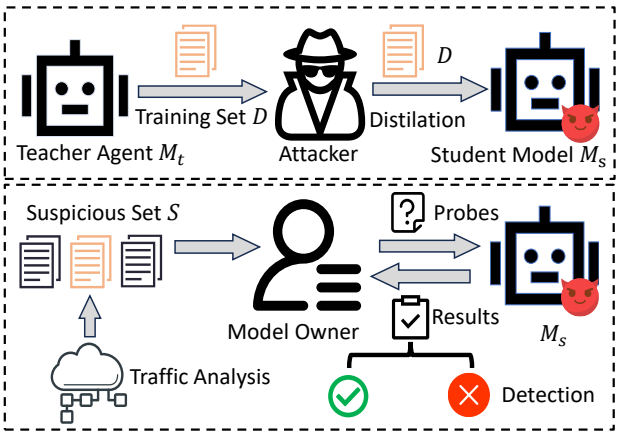


Figure 1: Overview of the access and threat model. The attacker distills a student from trajectories, while the model owner identifies suspicious traffic and audits the student.

We formulate a two-party threat model comprising a model owner and an attacker as shown in Figure 1. The owner deploys a teacher model $\mathcal{M}_t$, while the attacker aims to distill a student model $\mathcal{M}_s$ using a training dataset $\mathcal{D}$ collected from the teacher model.

Model Owner. The owner deploys and fully controls a teacher model $\mathcal{M}_t$ (e.g., Codex [24]) as a service. To protect the model, the owner embeds watermarks into the output trajectories in real time. Since data scraping has specific patterns, the owner monitors server-side features [9–11]. Any interactions showing such signs are recorded to build a suspicious trajectory set $\mathcal{S}$. To avoid missing potential distillation traces, the owner conservatively records all suspicious traces into $\mathcal{S}$ and the owner cannot determine which traces in $\mathcal{S}$ were actually used for distillation. When testing the deployed student model $\mathcal{M}_s$, the owner only has black box access and can fully control the API inputs to verify the watermark.

Attacker. To bypass the high training costs, the attacker aims to clone the capabilities of the teacher $\mathcal{M}_t$. Operating under strict black-box access, the attacker systematically queries the teacher model $\mathcal{M}_t$ to harvest complete multi-step interaction trajectories and this large-scale automated querying leaves identifiable server-side traffic patterns that can be captured by the model owner. The attacker may then preprocess these collected trajectories through operations such as sanitization, rewriting, truncation, or mixing with other data, forming the final distillation dataset $\mathcal{D}$. Finally, the attacker uses $\mathcal{D}$ to distill the student model $\mathcal{M}_s$ and deploys it as a black-box API.

3. Watermark Construction

3.1. Overview

During agent distillation, the distilled students and their teachers share similar interaction trajectories [17, 25, 26]. Based on this property, we proposed a watermark scheme `AuxMark` as illustrated in Figure 2. Firstly, the owner embeds the watermark by dynamically inserting auxiliary(aux) behaviors into the teacher's trajectories and privately saves evidence. Once suspicious traffic is identified, the corresponding records are extracted to form an evidence pool in Section 3.2. Next, the owner uses this evidence to construct paired real and fake probes, applying statistical tests to verify if the suspect model reproduces the watermark in Section 3.3. Finally, trace-level attribution in Section 3.4 to evaluate which suspicious trajectories are most likely to have been used in the attacker's training set.

3.2. Watermark Embedding Algorithm

Unlike previous watermarking methods that directly change the generated text or the teacher model's actions [15, 16], `AuxMark` embeds watermarks by adding extra, non-essential auxiliary actions into the agent's trajectory. By doing this, `AuxMark` keeps the teacher model's performance high while creating strong evidence for later tracing. Also, to stay hidden, the added aux actions look and act exactly

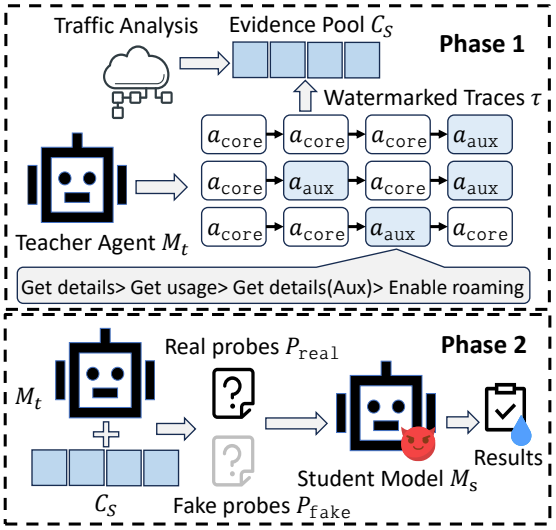


Figure 2: Overview of AuxMark. **Phase 1:** the owner embeds the watermark by inserting aux behaviors; **Phase 2:** the owner retrieves the private evidence to construct paired real and fake probes and verify a suspect student model. ■ denotes watermarked thing.

like normal tool calls: they use valid tools and bind strictly to normal parameter values derived from the interaction history. Based on the threat model described in Section 2, the owner can subsequently identify suspicious traces from the API traffic and export their privately saved watermark evidence for later verification. The entire embedding process consists of three main stages shown in Figure8: safe tool classification, dynamic scheduling and candidate generation, and private evidence retention.

Safe tool classification. To ensure aux actions will not change the environment state, AuxMark applies a conservative safety policy to the tool schema $\mathcal{F}$ of trace τ. Since the system operates as an agent-as-a-service [27], this safety evaluation is performed in advance and cached to avoid real-time latency. An offline safety model $\mathcal{M}_{\texttt{safe}}$ evaluates the tool schema $\mathcal{F}$ and maps each tool $f \in \mathcal{F}$ to a safety label as shown in Eq.2:

$$\mathcal{M}_{\texttt{safe}}(f) = \begin{cases} \texttt{safe}, & \text{if } f \text{ is read-only,} \\ \texttt{unsafe}, & \text{otherwise.} \end{cases} \tag{2}$$

The system exclusively permits tools from the valid subset $\mathcal{F}_{\texttt{safe}} = \{f \in \mathcal{F} \mid \mathcal{M}_{\texttt{safe}}(f) = \texttt{safe}\}$, categorically rejecting any action that changes the environment state or influences the task process.

Dynamic scheduling. Having established the safe tool subset $\mathcal{F}_{\texttt{safe}}$, the system must determine when to invoke these aux action during the interaction as the watermarking. To make watermark positions unpredictable, AuxMark dynamically evaluates whether to inject an aux action at step t immediately after the core action $a_{\texttt{core}}^{t-1}$ finishes. If skipped, the system waits and checks again after the next core action. For each trajectory τ named as $\texttt{id}_\tau$ of an account a, the system keeps a release probability p_t and a remaining budget b_t at step t. Immediately after the core action $a_{\texttt{core}}^{t-1}$ finishes, the system uses a cryptographic hash function $\texttt{Hash}(\cdot) \to [0,1)$ [28] parameterized by the owner's secret key k to compute a uniformly distributed random value u_t, as defined in Eq. 3:

$$u_t = \texttt{Hash}(k \parallel a \parallel \texttt{id}_\tau \parallel t) \in [0,1). \tag{3}$$

The system tries to add an aux action when $b_t > 0$ and $u_t < p_t$. Depending on whether the auxiliary action is successfully inserted or the turn is skipped (due to generation failure or unmet conditions), the system updates the trigger probability and remaining budget for the next turn as follows:

$$(p_{t+1}, b_{t+1}) = \begin{cases} (p_0, b_t - 1), & \text{if successfully inserted,} \\ (\min\{p_t + \Delta p, p_{\max}\}, b_t), & \text{otherwise.} \end{cases} \tag{4}$$

Aux action embedding. If the dynamic scheduling condition is met, AuxMark proceeds to generate and select the optimal aux action. To obtain high quality insertions, the system queries $\mathcal{M}_t$ to propose up to K candidates. Each candidate j is formulated as a tuple of a thought and an action: $(t_{\texttt{aux}}^j, a_{\texttt{aux}}^j)$, where the action $a_{\texttt{aux}}^j$ consists of a tool $f_{\texttt{aux}}^j$ and its arguments $p_{\texttt{aux}}^j$. To ensure that the injected action perfectly blends into the trajectory, AuxMark enforces a validation mechanism. First, the chosen tool must belong to the safe subset $\mathcal{F}_{\texttt{safe}}$. Second, to prevent model hallucinations and ensure contextual rationality, the arguments $p_{\texttt{aux}}^j$ must be extracted from the context (such as the core action's observation or history). Consequently, AuxMark categorically discards any candidate that uses an unapproved tool or hallucinates untraceable arguments.

For each candidate j that passes the validation, AuxMark employs a scoring model $\mathcal{M}_q$ to evaluate it across three dimensions: naturalness, logical consistency, and distillability. Specifically, $\mathcal{M}_q$ assigns a relevance score $s_{\texttt{rel}}^j$ to ensure the injected action aligns naturally with the task. Additionally, a reliability score $s_{\texttt{relb}}^j$ ensures the high quality of the thought. Crucially, to foster a stable behavioral habit that the student model can easily absorb during distillation, the system calculates a repeatability score $s_{\texttt{rep}}^j$. This score is directly determined by n_j, which represents the number of times the candidate's specific tool pairing $(f_{\texttt{core}}, f_{\texttt{aux}}^j)$ has been successfully injected in the account's history. The final score S_j for candidate j is computed as a weighted sum:

$$S_j = \alpha \cdot s_{\texttt{rel}}^j + \beta \cdot s_{\texttt{relb}}^j + \gamma \cdot s_{\texttt{rep}}^j. \tag{5}$$

where α, β, γ, and the step-function mapping from repeat count n_j to $s_{\texttt{rep}}^j$ are detailed in Appendix B.1. The system selects the candidate with the highest score as the best aux action $(t_{\texttt{aux}}^{\texttt{best}}, a_{\texttt{aux}}^{\texttt{best}})$, then injects this aux action, establishing a natural sequence:

$$t_{\texttt{core}}^{t-1} \to a_{\texttt{core}}^{t-1} \to o_{\texttt{core}}^{t-1} \to t_{\texttt{aux}}^{\texttt{best}} \to a_{\texttt{aux}}^{\texttt{best}} \to o_{\texttt{aux}}^{\texttt{best}}. \tag{6}$$

Based on the generation result, the system updates its states. A successful generation applies the success update (Eq. 4)

and increases the repeat count $n_{\texttt{best}}$ for this tool pairing. If the generation fails, the system skips the insertion, applies the failure update, moves to generate core action and waits for next chance. For future auditing, every successful insertion creates a private evidence card c_t. To securely record the exact context and execution details, this card stores the historical context h, the preceding core action and observation, and the injected auxiliary tool parameters. It is saved as a tuple and added to the trajectory's evidence set $\mathcal{C}_\tau$:

$$c_t = (h, t^{t-1}_{\texttt{core}}, a^{t-1}_{\texttt{core}}, o^{t-1}_{\texttt{core}}, f^{\texttt{best}}_{\texttt{aux}}, p^{\texttt{best}}_{\texttt{aux}}), \quad \mathcal{C}_\tau = \mathcal{C}_\tau \cup \{c_t\}. \tag{7}$$

As mentioned in our threat model, the model owner uses signals like querying styles, IPs, and traffic patterns to identify a suspicious set $\mathcal{S}$ [9, 11]. The owner then collects the evidence from these flagged traces to form a global verification pool $\mathcal{C}_\mathcal{S} = \bigcup_{\tau \in \mathcal{S}} \mathcal{C}_\tau$.

3.3. Watermark Detection

While the embedding phase injects an aux action into the teacher's trajectory, the detection phase verifies if a suspect model M_s has learned this specific behavior pattern. To achieve this, AuxMark uses paired probes built from the combined evidence pool $\mathcal{C}_\mathcal{S}$ for verification as shown in Figure 9.

Paired probe construction. To verify watermark retention, AuxMark constructs paired probes for the combined evidence pool $\mathcal{C}_\mathcal{S}$. A probe is formally defined as a prompt-target tuple: given a history and a core action as the prompt, the suspect model $\mathcal{M}_s$ is expected to output a specific aux action. Specifically, based on each evidence card, we derive two types of probes: real probes and fake probes. This paired design ensures that the model $\mathcal{M}_s$ has genuinely memorized the specific watermarked trace, rather than unconditionally outputting the target action.

For the real side construction of each card $c_i \in \mathcal{C}_S$, the system first locates the nearest core action. To ensure that the suspect model M_s has learned the expected behavior, AuxMark creates up to N parameterized variants for this real side by modifying one argument of the original core action, the system prompts the teacher model $\mathcal{M}_t$ to synchronously generate a logically consistent core triplet $(t^j_{\texttt{core}}, a^j_{\texttt{core}}, o^j_{\texttt{core}})$ alongside the new downstream target aux arguments $p^j_{\texttt{aux}}$. This process directly generates a set of real side probes $p^i_{\texttt{real}}$ for the specific card c_i as shown in Eq.8:

$$p^i_{\texttt{real}} = \left\{(h, t^j_{\texttt{core}}, a^j_{\texttt{core}}, o^j_{\texttt{core}}),\ (f_{\texttt{aux}}, p^j_{\texttt{aux}})\right\}_{j=1}^{N_i}. \tag{8}$$

h is the historical context, and $f_{\texttt{aux}}$ is the targeted aux tool. Finally, the system aggregates the real-side probes from all cards to form the global real probe set $\mathcal{P}_{\texttt{real}} = \bigcup_{c_i \in \mathcal{C}_S} p^i_{\texttt{real}}$.

To better contrast with the real probes $\mathcal{P}_{\texttt{real}}$, obtain more definitive statistical results, AuxMark constructs a set of fake probes based on the $\mathcal{P}_{\texttt{real}}$. This verifies whether the aux action stems from memorization rather than natural context or random chance.For this fake side. However, it prompts the teacher model $\mathcal{M}_t$ to replace the core action with a semantically similar fake core behavior $(\hat{t}^j_{\texttt{core}}, \hat{a}^j_{\texttt{core}}, \hat{o}^j_{\texttt{core}})$ that achieves the identical goal. This process generates the fake probe set $p^i_{\texttt{fake}}$ for card c_i, directly matching the variants in $p^i_{\texttt{real}}$ as shown in Eq.9:

$$p^i_{\texttt{fake}} = \left\{(h, \hat{t}^j_{\texttt{core}}, \hat{a}^j_{\texttt{core}}, \hat{o}^j_{\texttt{core}}),\ (f_{\texttt{aux}}, p^j_{\texttt{aux}})\right\}_{j=1}^{N_i}. \tag{9}$$

Thus, the strict difference between the paired probes lies only in the core action. Similar to the real side, the fake probe set is defined as $\mathcal{P}_{\texttt{fake}} = \bigcup_{c_i \in \mathcal{C}_S} p^i_{\texttt{fake}}$.

Card-level statistical test. After generating the paired probes, AuxMark evaluates the suspect model M_s using $(\mathcal{P}_{\texttt{real}}, \mathcal{P}_{\texttt{fake}})$ and performs a card-level statistical test which is discussed in Appendix C. For the j-th probe pair of card c_i, the expected aux action is $f_{\texttt{aux}}$ and arguments $p^j_{\texttt{aux}}$. Let $(f_{\texttt{out}}, p_{\texttt{out}})$ be the tool call parsed from the output of M_s. For both the real and fake sides, AuxMark computes the toolhit indicator $t_{i,j}$ and the strict fullhit indicator $h_{i,j} \in \{0, 1\}$:

$$t_{i,j} = \texttt{Match}(f_{\texttt{out}}, f_{\texttt{aux}}), \quad h_{i,j} = t_{i,j} \cdot \texttt{Match}(p_{\texttt{out}}, p^j_{\texttt{aux}}). \tag{10}$$

where the function $\texttt{Match}(\cdot, \cdot) \to \{0, 1\}$ evaluates to 1 if and only if there is exact structural and value equivalence between its two arguments, and 0 otherwise. Because the N_i paired variants derived from the same card c_i share an identical core-to-aux dependency, they are not independent samples. Therefore, AuxMark aggregates the scores at the card level to calculate the real-side hit rate $R^{\texttt{real}}_i$, the fake-side hit rate $R^{\texttt{fake}}_i$, and their difference Δ_i for each card c_i as shown in Eq.11:

$$R^{\texttt{real}}_i = \frac{1}{N_i}\sum_{j=1}^{N_i} h^{\texttt{real}}_{i,j}, \qquad R^{\texttt{fake}}_i = \frac{1}{N_i}\sum_{j=1}^{N_i} h^{\texttt{fake}}_{i,j}, \qquad \Delta_i = R^{\texttt{real}}_i - R^{\texttt{fa}}_i \tag{11}$$

To evaluate the verification, we establish the null hypothesis H_0 and the alternative hypothesis H_1. H_0 posits that the suspect model $\mathcal{M}_s$ has not learned the watermarked habit; any correct action is due to general context or innate biases. Conversely, H_1 asserts that $\mathcal{M}_s$ has memorized the watermark, making it significantly more likely to output the target action on real probes. Based on the performance difference Δ_i for each card c_i, we classify the outcomes into a real win ($\Delta_i > 0$), a fake win ($\Delta_i < 0$), or a tie ($\Delta_i = 0$). Excluding ties, let w and ℓ be the numbers of real and fake wins across $\mathcal{C}_\mathcal{S}$. The p-value is computed using a one-sided exact sign-test [29]:

$$p{-}\texttt{value} = \sum_{k=w}^{w+\ell} \binom{w+\ell}{k} 2^{-(w+\ell)}. \tag{12}$$

where $p{-}\texttt{value} = 1$ if $w + \ell = 0$. To reject H_0 and declare a successful verification, AuxMark requires dual criteria: the statistical significance must be $p{-}\texttt{value} < p$, and the averaged real fullhit rate must exceed the fake

rate by a predefined margin r (i.e., $\frac{1}{|\mathcal{C}_\mathcal{S}|}\sum \Delta_i \geq r$). This empirical margin absorbs unpredictable fluctuations caused by context rewriting in the fake probes and guards against false positives. Appendix B.1 specifies the hyperparameters p, r, and N.

3.4. Trace-Level Attribution

While the detection protocol in Section 3.3 determines whether a suspect model $\mathcal{M}_s$ has distilled from $\mathcal{M}_t$, it cannot pinpoint which traces within $\mathcal{S}$ were utilized. For exact attribution, `AuxMark` introduces a trace-level attribution. For each evaluated trace τ, the system computes four rates to capture both exact and partial matches: the strict full-hit rates $R_\tau^{\texttt{real}}$ and $R_\tau^{\texttt{fake}}$, and the toolhit-only rates $T_\tau^{\texttt{real}}$ and $T_\tau^{\texttt{fake}}$ (where $t_{i,j} = 1 \wedge h_{i,j} = 0$). To ensure fair comparison across traces, `AuxMark` normalizes each rate into an average-rank quantile $Q(\cdot) \in (0, 1)$:

$$Q(x_\tau) = \frac{\texttt{rank}_{\texttt{avg}}(x_\tau)}{|\mathcal{S}| + 1}. \tag{13}$$

The attribution logic is designed as follows: $Q(R_\tau^{\texttt{real}})$ acts as the primary positive signal for trace memorization. Conversely, if the model outputs the target aux action even on the fake probes, it implies innate biases rather than specific watermark retention. Thus, $Q(R_\tau^{\texttt{fake}})$ serves as a penalty term. Furthermore, the toolhit-only terms address partial memorization, providing supplementary evidence to calibrate the final score. Consequently, `AuxMark` ranks these traces using a score S_τ:

$$S_\tau = Q(R_\tau^{\texttt{real}}) - \alpha' Q(R_\tau^{\texttt{fake}}) + \beta' Q(T_\tau^{\texttt{real}}) - \gamma' Q(T_\tau^{\texttt{fake}}). \tag{14}$$

where the empirical weighting parameters α', β', and γ' are detailed in Appendix B.1. Finally, the system identifies the top-K traces with the highest S_τ most likely included in the training set $\mathcal{D}$.

4. Experiments

4.1. Experimental Setup

We evaluate `AuxMark` in agent distillation. We supervise thoughts and actions from the trajectories and fine-tune the student models using LoRA. The specific training configurations are rank 32, learning rate 2×10^{-4}, and $\alpha = 64$. Prior traffic-analysis systems have reported identification precision above 70% [9, 11]. We therefore adopt a more conservative 50% precision setting to avoid missing potential distillation traces, with $|\mathcal{S}| = 100$ suspicious trajectories and $|\mathcal{D}_c| = 50$ trajectories used for distillation. Appendix B.1 provides the remaining experimental details, including hyperparameter selection and baselines. Our experiments assess watermark effectiveness, harmlessness, robustness against adversarial processing.

Datasets and Models. Our evaluation spans three datasets: BFCL [30], SWE-bench [31], and Telecom (a service-tool domain from the Tau2 benchmark[1]). GPT-OSS-120B [32] and Kimi-K2.5 [33] serve as teacher agents ($\mathcal{M}_t$), with Qwen3-8B acting as both the candidate scorer ($\mathcal{M}_q$) and safety model ($\mathcal{M}_{\texttt{safe}}$). For distillation, we fine-tune four student models ($\mathcal{M}_s$): Mistral-Small-3.1-24B [34], Qwen3-14B [35], Qwen3-32B [35], and GLM-4.7-Flash [36].

Evaluation metrics. For detection, we report strict fullhit rates $(R_\tau^{\texttt{real}}, R_\tau^{\texttt{fake}})$ on real and fake probes, alongside the one-sided p-value. A successful detection requires $p < 0.05$ and a real-fake mean fullhit gap ≥ 0.05. For trace-level attribution, we measure `Precision`@K of top-K trajectories belonging to the distillation subset $\mathcal{D}_c$. For harmlessness, we compare task accuracy (`Acc`) and mean interaction steps (L) between control and watermarked agents.

4.2. Effectiveness

In this section, all student models are trained on the original dataset $\mathcal{D}_c$. We test our scheme on distilled and clean models via separate true positive and false positive experiments. We also evaluate the performance of existing agent watermark baselines under the matched setting. Furthermore, we evaluate `AuxMark`'s trace-level attribution using `Precision`@K in Appendix B.2. Finally, we measure how identification accuracy affects statistical significance in Appendix B.2.

True positives. We first evaluate whether `AuxMark` can successfully identify student models distilled from watermarked trajectories. Our experiments cover all combinations of 3 datasets (SWE and Telecom are in Appendix B.2), 2 teacher agents, and 4 student models. In each setting, we compare the hit rates between the real side and the fake side and calculate the p-value. The results in the Table 1 and 6 show that `AuxMark` successfully achieves detection in all settings. These results show that `AuxMark` can extract watermark from student models of different architectures.

False positives. We evaluate whether `AuxMark` causes false positives on clean models that have not seen watermarked trajectories. As shown in Table 2 and Table 7 in Appendix B.2, in addition to the four undistilled student models, we also select four base models (DeepSeek-V4-Flash [37], MiniMax-M2.5 [38], Qwen3.5-Flash [35], and MiMo-V2.5 [39]) and build a total of 48 controlled false-positive evaluation settings. The results show that `AuxMark` achieves zero false positives across all 48 settings.

Baseline effectiveness. We evaluate the effectiveness of the two baseline schemes under the matched distillation setting and the details about baselines are in Appendix B.1. As shown in Table 3, for Agentwm, only the GLM-4.7-Flash student model meets the passing criterion of $\geq 3/5$ passes, while all other students fail. For Seqwm, the median p-values across all tested models are greater than 0.05, failing to successfully detect any distilled models. As expected, baselines fail, because Seqwm is not designed for antidistillation, and Agentwm has weak watermark signals that requires identical teacher and student base models.

1. https://github.com/sierra-research/tau2-bench

TABLE 1: True-positive results. Cards is the number of cards; hit-rate cells report hits / probes (rate). w/ℓ gives card-level real / fake wins. The 'Sig' column indicates statistical significance: ✓ denotes $p < 0.05$, indicating a statistically significant change, whereas × denotes $p \geq 0.05$.

Teacher Agent $\mathcal{M}_t$	Dataset	Cards	Student Model $\mathcal{M}_s$	Detection Results				
				$R^{\texttt{real}}$ (%)	$R^{\texttt{fake}}$ (%)	w/ℓ	p-value	Sig.
GPT-OSS-120B	BFCL	118	Mistral-24B	93 / 182 (51.1%)	59 / 182 (32.4%)	25 / 1	4.0×10^{-7}	✓
			GLM-4.7-Flash	108 / 182 (59.3%)	80 / 182 (44.0%)	19 / 1	2.0×10^{-5}	✓
			Qwen3-14B	44 / 182 (24.2%)	28 / 182 (15.4%)	20 / 4	7.7×10^{-4}	✓
			Qwen3-32B	89 / 182 (48.9%)	58 / 182 (31.9%)	30 / 9	5.3×10^{-4}	✓
Kimi-K2.5	BFCL	145	Mistral-24B	175 / 379 (46.2%)	105 / 379 (27.7%)	45 / 10	1.0×10^{-6}	✓
			GLM-4.7-Flash	155 / 379 (40.9%)	69 / 379 (18.2%)	47 / 4	1.2×10^{-10}	✓
			Qwen3-14B	102 / 379 (26.9%)	26 / 379 (6.9%)	46 / 4	2.2×10^{-10}	✓
			Qwen3-32B	82 / 379 (21.6%)	18 / 379 (4.7%)	39 / 5	7.0×10^{-8}	✓

TABLE 2: False-positive results. The 'Sig' column indicates statistical significance: ✓ denotes $p < 0.05$, indicating a statistically significant change, whereas × denotes $p \geq 0.05$.

Teacher Agent $\mathcal{M}_t$	Dataset	Cards	Clean Model $\mathcal{M}_c$	Detection Results				
				$R^{\texttt{real}}$ (%)	$R^{\texttt{fake}}$ (%)	w/ℓ	p-value	Sig.
GPT-OSS-120B	BFCL	118	DeepSeek-V4-Flash	10 / 182 (5.5%)	11 / 182 (6.0%)	2 / 3	8.1×10^{-1}	×
			MiniMax-M2.5	9 / 182 (4.9%)	10 / 182 (5.5%)	2 / 3	8.1×10^{-1}	×
			Mistral-24B	9 / 182 (4.9%)	7 / 182 (3.8%)	3 / 2	5.0×10^{-1}	×
			Qwen3-14B	7 / 182 (3.8%)	6 / 182 (3.3%)	2 / 1	5.0×10^{-1}	×
			Qwen3-32B	4 / 182 (2.2%)	4 / 182 (2.2%)	2 / 2	6.9×10^{-1}	×
			Qwen3.5-Flash	8 / 182 (4.4%)	8 / 182 (4.4%)	2 / 1	5.0×10^{-1}	×
			MiMo-V2.5	10 / 182 (5.5%)	11 / 182 (6.0%)	2 / 2	6.9×10^{-1}	×
			GLM-4.7-Flash	5 / 182 (2.7%)	1 / 182 (0.5%)	4 / 0	6.2×10^{-2}	×
Kimi-K2.5	BFCL	145	DeepSeek-V4-Flash	10 / 379 (2.6%)	9 / 379 (2.4%)	5 / 4	5.0×10^{-1}	×
			MiniMax-M2.5	6 / 379 (1.6%)	3 / 379 (0.8%)	4 / 2	3.4×10^{-1}	×
			Mistral-24B	1 / 379 (0.3%)	3 / 379 (0.8%)	1 / 2	8.8×10^{-1}	×
			Qwen3-14B	1 / 379 (0.3%)	0 / 379 (0.0%)	1 / 0	5.0×10^{-1}	×
			Qwen3-32B	1 / 379 (0.3%)	0 / 379 (0.0%)	1 / 0	5.0×10^{-1}	×
			Qwen3.5-Flash	0 / 379 (0.0%)	0 / 379 (0.0%)	0 / 0	1.0×10^{0}	×
			MiMo-V2.5	6 / 379 (1.6%)	6 / 379 (1.6%)	4 / 3	5.0×10^{-1}	×
			GLM-4.7-Flash	2 / 379 (0.5%)	2 / 379 (0.5%)	2 / 1	5.0×10^{-1}	×

TABLE 3: Baseline effectiveness. Agentwm is significant at $\geq 3/5$ passes; SeqWM at median $p < 0.05$.

Dataset	Student Model $\mathcal{M}_s$	Agentwm		SeqWM	
		Passes	Sig.	Median p	Sig.
BFCL	Mistral-24B	1/5	×	0.6848	×
	GLM-4.7-Flash	3/5	✓	0.5130	×
	Qwen3-14B	1/5	×	0.4650	×
	Qwen3-32B	2/5	×	0.5744	×
SWE-bench	Mistral-24B	1/5	×	0.6274	×
	GLM-4.7-Flash	3/5	✓	0.5914	×
	Qwen3-14B	1/5	×	0.7073	×
	Qwen3-32B	0/5	×	0.4895	×

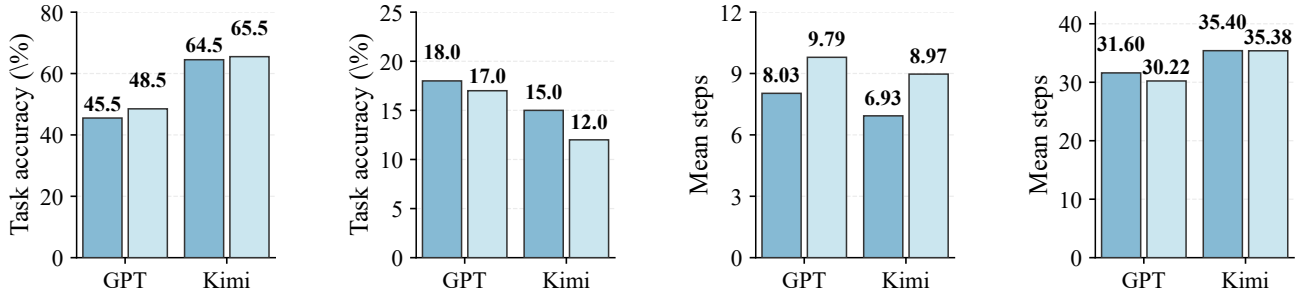
4.3. Harmlessness

Harmlessness. A reasonable watermark scheme should not noticeably affect the performance of the teacher agent. Therefore, we evaluate impact of AuxMark on the teacher agent $\mathcal{M}_t$'s task utility and inference cost. For the four cohorts in BFCL and Telecom, we compare the unwatermarked agent $\mathcal{M}_{\texttt{tc}}$ and the watermarked teacher agent $\mathcal{M}_t$ on 200 tasks. We measure the task accuracy difference $\Delta\texttt{Acc} = \mathbb{E}[\texttt{Acc}(\mathcal{M}_t) - \texttt{Acc}(\mathcal{M}_{\texttt{tc}})]$ and the expected interaction step difference $\Delta L = \mathbb{E}[L(\mathcal{M}_t) - L(\mathcal{M}_{\texttt{tc}})]$. As summarized in Figure 3, embedding the watermark does not cause a drop in overall performance. The task accuracy difference $\Delta\texttt{Acc}$ ranges only from -3.0% to $+3.0\%$, indicating that the accuracy loss is acceptable. Meanwhile, the increase in interaction cost is acceptable: the average step count increases from 20.49 to 21.09 steps, and ΔL ranges from -1.38 to $+2.04$ steps.

4.4. Robustness

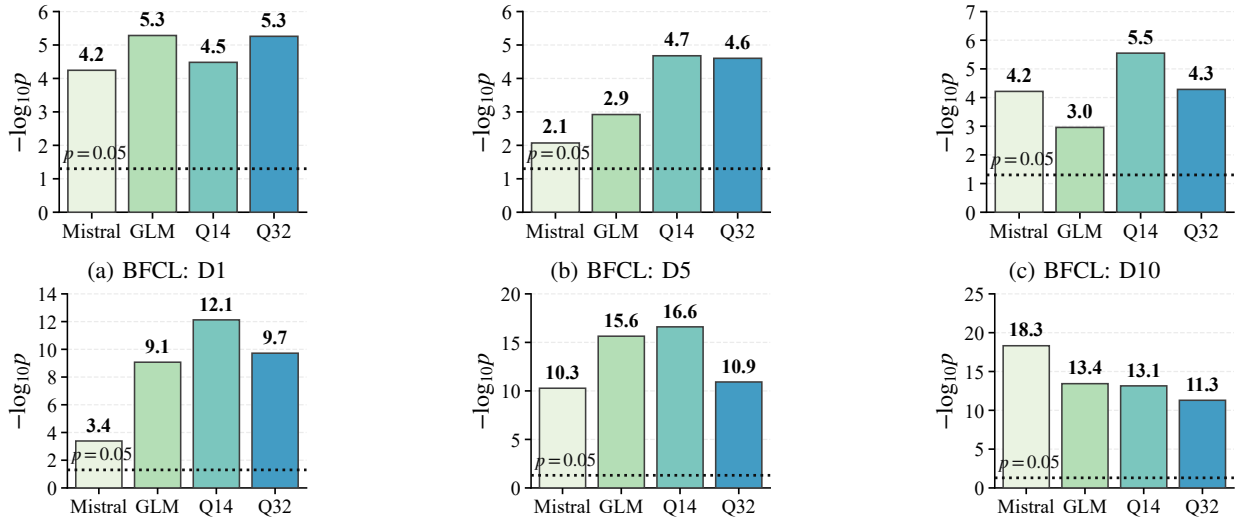
To demonstrate the robustness of our watermark scheme, We evaluate AuxMark's robustness against four common pre-distillation data transformations. These operations include: data flooding, paraphrasing attack, adaptive attack, advanced data flooding and truncation (the last two are in Appendix B.3). We evaluate the robustness of Agentwm under data flooding in Appendix B.3. We use GPT-OSS-120B as the teacher agent.

Data flooding. Before distillation, an attacker may mix the training set with standard clean data to improve distillation performance [3, 23], which will dilutes the watermark signals. To evaluate its impact on our watermark strength, we mix the original watermarked dataset $\mathcal{D}_c$ with clean standard answers. Specifically, in the D1, D5, and D10 settings, we incorporate $1\times$, $5\times$, and $10\times$ the amount of clean data relative to $\mathcal{D}_c$. We evaluate a total of 24 settings across 2 benchmarks (BFCL and SWE-bench), 3 dilution ratios, and 4 student models. As shown in Figure 4, AuxMark successfully detects every setting. Even under the strongest 1:10 dilution ratio, all 8 tested student models remain successfully detected.



(a) BFCL: accuracy (b) Telecom: accuracy (c) BFCL: steps (d) Telecom: steps

Figure 3: Harmlessness results. In (a) and (b), the vertical axis reports accuracy; in (c) and (d), it reports the mean number of steps. Colors distinguish unwatermarked and watermarked runs.



(a) BFCL: D1 (b) BFCL: D5 (c) BFCL: D10

(d) SWE-bench: D1 (e) SWE-bench: D5 (f) SWE-bench: D10

Figure 4: Data-flooding robustness. The vertical axis reports $-\log_{10} p$. In (a)–(f), bar colors denote Mistral-24B, GLM-4.7-Flash, Qwen3-14B, and Qwen3-32B. The ···· marks $p = 0.05$.

TABLE 4: Paraphrasing attack robustness.

Dataset	Student Model $\mathcal{M}_s$	p-value	Sig.
BFCL	Mistral-24B	5.7×10^{-5}	✓
	GLM-4.7-Flash	1.4×10^{-4}	✓
	Qwen3-14B	3.3×10^{-3}	✓
	Qwen3-32B	9.8×10^{-5}	✓
Telecom	Mistral-24B	3.2×10^{-2}	✓
	GLM-4.7-Flash	1.4×10^{-5}	✓
	Qwen3-14B	3.5×10^{-3}	✓
	Qwen3-32B	4.1×10^{-3}	✓

TABLE 5: Adaptive attack robustness.

Dataset	Student Model $\mathcal{M}_s$	p-value	Sig.
BFCL	Mistral-24B	9.2×10^{-4}	✓
	GLM-4.7-Flash	1.4×10^{-4}	✓
	Qwen3-14B	9.6×10^{-3}	✓
	Qwen3-32B	4.1×10^{-4}	✓
Telecom	Mistral-24B	2.8×10^{-8}	✓
	GLM-4.7-Flash	3.7×10^{-3}	✓
	Qwen3-14B	1.0×10^{-11}	✓
	Qwen3-32B	5.5×10^{-12}	✓

Paraphrasing attack. An attacker may paraphrase the trajectories, aiming to alter semantic patterns and evade watermark detection while preserving the utility [40]. To evaluate the robustness against this rewriting attack, we use DeepSeek-V4-Flash to paraphrase the $\mathcal{D}_c$. Subsequently, we train 4 student models on these rewritten datasets across the BFCL and Telecom and detect their watermark signals. As reported in Table 4, `AuxMark` successfully detects the watermark across all 8 settings showing our scheme's robustness under paraphrasing.

Adaptive attack. In this section, we consider a stronger attacker who knows that our watermark relies on auxiliary behaviors. To avoid detection, the attacker uses DeepSeek-V4-Flash to clean the collected data by finding and removing suspected auxiliary action while keeping the trajectories useful. Next, we fine-tune student models on this cleaned data and test them on BFCL and Telecom. As shown in Table 5, even with this removal, `AuxMark` successfully

detects all student models (with the minimal p-value of 9.6×10^{-3}). This shows that our watermark is highly robust, even when an attacker actively tries to move the signals.

5. Conclusion

We present AuxMark, a behavioral watermarking framework to track the unauthorized distillation of LLM agents. AuxMark inserts useful extra tool calls after normal core actions. To verify if the watermark is copied, it uses paired real and fake probes along with a card-level sign test. This design supports both model-level detection and trace-level tracking.In experiments across three benchmarks, two teacher models, and four student architectures, AuxMark successfully detects all 24 distilled students with zero false positives on 48 clean models. It also remains highly effective under data flooding, rewriting, truncation, and adaptive cleaning attacks. Future work will extend this evaluation to more teacher models, cleaning tools, and training methods.

AI use statement

In this work, we used generative AI tools for research execution, generating synthetic datasets, and code generation. Specifically, large language models were employed to generate agent-interaction trajectories, construct paired verification probes for our evaluations, and assist with implementation. We have not used generative AI tools for drafting core sections of the paper or proving mathematical claims, and the rest of the required disclosure tasks are not applicable to this work. Additionally, we used generative AI tools (Gemini 3.1 Pro) for aiding and polishing the English writing of the manuscript. We have reviewed all AI-assisted work. The authors manually assessed AI-assisted research ideas through a literature survey and technical discussion. For the synthetic datasets, the LLM-generated agent trajectories and verification probes were programmatically validated and manually sampled to ensure they form valid, executable tool-use sequences that strictly adhere to the predefined tool schemas and task objectives. All AI-generated code was reviewed, tested, and, where necessary, revised by the authors before use. For the writing assistance, all AI-polished text was thoroughly reviewed and edited by the authors to guarantee it accurately reflects our original scientific intent, methodology, and conclusions without introducing hallucinations or overclaims. We take responsibility for the final content of this work, including text, claims, or artifacts produced with the aid of generative AI.

Reproducibility Statement

To ensure the full reproducibility of our results, we have made our complete source code, benchmark adapters, and evaluation scripts publicly available via an anonymous repository (https://github.com/qx041609/Auxmark). Additionally, all trained student model weights (e.g., LoRA adapters) necessary to replicate our detection and trace-level attribution experiments are hosted on Hugging Face (huggingface.co/AuxMark/AuxMark/tree/main).

References

[1] G. Hinton, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, vol. 2, 2015.

[2] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhost, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, and T. Pfister, "Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes," in *Findings of the association for computational linguistics: ACL 2023*, 2023, pp. 8003–8017.

[3] S. Mukherjee, A. Mitra, G. Jawahar, S. Agarwal, H. Palangi, and A. Awadallah, "Orca: Progressive learning from complex explanation traces of gpt-4," *arXiv preprint arXiv:2306.02707*, 2023.

[4] J. Liu, Z. Kong, P. Dong, C. Yang, T. Li, Y. Xie, Y. Gong, X. Shen, P. Zhao, H. Tang *et al.*, "Structured agent distillation for large language model agents," in *Proc. of the 25th International Conference on Autonomous Agents and Multiagent Systems*, 2026, pp. 3676–3685.

[5] M. Kang, J. Jeong, S. Lee, J. Cho, and S. J. Hwang, "Distilling llm agent into small models with retrieval and code tools," *Advances in Neural Information Processing Systems*, vol. 38, pp. 106 501–106 538, 2026.

[6] Anthropic, "Detecting and preventing distillation attacks," https://www.anthropic.com/news/detecting-and-preventing-distillation-attacks, 2026, [Accessed 17-09-2026].

[7] OpenAI, "Updated Stakes for American-Led, Democratic AI," https://assets.bwbx.io/documents/users/iqjWHBFdfxIU/rRmql_jJcxb4/v0, 2026, [Accessed 17-09-2026].

[8] G. T. I. Group, "Gtig ai threat tracker: Distillation, experimentation, and (continued) integration of ai for adversarial use," https://cloud.google.com/blog/topics/threat-intelligence/distillation-experimentation-integration-ai-adversarial-use, 2026, [Accessed 17-09-2026].

[9] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez *et al.*, "Chatbot arena: An open platform for evaluating llms by human preference," *arXiv preprint arXiv:2403.04132*, 2024.

[10] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, and N. Choi, "Preventing artificially inflated {SMS} attacks through {Large-Scale} traffic inspection," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 5405–5423.

[11] S. Liu, Q. Guo, and Y. Dong, "An embarrassingly simple detector for model extraction attacks in large language model api traffic," *arXiv preprint arXiv:2606.05725*, 2026.

[12] Y. Savani, A. Trockman, Z. Feng, Y. Xu, A. Schwarzschild, A. Robey, M. Finzi, and Z. Kolter,

"Antidistillation sampling," *Advances in Neural Information Processing Systems*, vol. 38, pp. 117 800–117 827, 2026.

[13] Y. E. Xu, J. Kirchenbauer, Y. Savani, A. Trockman, A. Robey, T. Goldstein, F. Fang, and J. Z. Kolter, "Antidistillation fingerprinting," *arXiv preprint arXiv:2602.03812*, 2026.

[14] X. Ma, W. Yeoh, N. Zhang, and Y. Vorobeychik, "Protecting language models against unauthorized distillation through trace rewriting," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 11 307–11 324.

[15] L. Wang, Z. Li, Y. Xie, S. Wang, D. She, W. Wang, and J. Rahmel, "On protecting agentic systems' intellectual property via watermarking," *arXiv preprint arXiv:2602.08401*, 2026.

[16] H. An, S. Park, D. Kim, and Y.-S. Han, "Sequential behavioral watermarking for llm agents," *arXiv preprint arXiv:2605.11036*, 2026.

[17] C. Yang, Y. Zhang, Z. Wen, T. Gong, J. Liu, Q. Chu, and N. Yu, "When agents look the same: Quantifying distillation-induced similarity in tool-use behaviors," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 10 482–10 502.

[18] R. Rawat, S. Chen, A. Anand, M. Duan, B. Rotsted, and S. Min, "Reference-based distillation detection in llms," *arXiv preprint arXiv:2607.09692*, 2026.

[19] A. Panfilov, D. Schmotz, I. Shumailov, L. Beurer-Kellner, J. Schaeffer, A. Prabhu, J. Geiping, and M. Andriushchenko, "Stealing reasoning traces from proprietary llm apis," *arXiv preprint arXiv:2608.09867*, 2026.

[20] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[21] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, "Toolllm: Facilitating large language models to master 16000+ real-world apis," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 9695–9717.

[22] Y. Luo, Y. Jin, W. Yu, M. Zhang, S. Kumar, X. Li, W. Xu, X. Chen, and J. Wang, "Agentark: Distilling multi-agent intelligence into a single llm agent," *arXiv preprint arXiv:2602.03955*, 2026.

[23] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.

[24] A. Singh, A. Fry, A. Perelman, A. Tart, A. Ganesh, A. El-Kishky, A. McLaughlin, A. Low, A. Ostrow, A. Ananthram *et al.*, "Openai gpt-5 system card," *arXiv preprint arXiv:2601.03267*, 2025.

[25] Y. Lyu, C. Wang, J. Huang, and T. Xu, "From correction to mastery: Reinforced distillation of large language model agents," *arXiv preprint arXiv:2509.14257*, 2025.

[26] A. Gudibande, E. Wallace, C. Snell, X. Geng, H. Liu, P. Abbeel, S. Levine, and D. Song, "The false promise of imitating proprietary llms," *arXiv preprint arXiv:2305.15717*, 2023.

[27] H. Li, H. Zhang, and A. E. Hassan, "Aidev: studying ai coding agents on github," in *Proceedings of the 23rd International Conference on Mining Software Repositories*, 2026, pp. 1029–1033.

[28] B. Preneel, "Cryptographic hash functions," *European Transactions on Telecommunications*, vol. 5, no. 4, pp. 431–448, 1994.

[29] M. Hollander, D. A. Wolfe, and E. Chicken, *Nonparametric statistical methods*. John Wiley & Sons, 2013.

[30] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica, and J. E. Gonzalez, "The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models," in *Forty-second International Conference on Machine Learning*, 2025.

[31] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "Swe-bench: Can language models resolve real-world github issues?" in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 54 107–54 157.

[32] S. Agarwal, L. Ahmad, J. Ai, S. Altman, A. Applebaum, E. Arbus, R. K. Arora, Y. Bai, B. Baker, H. Bao *et al.*, "gpt-oss-120b & gpt-oss-20b model card," *arXiv preprint arXiv:2508.10925*, 2025.

[33] K. Team, T. Bai, Y. Bai, Y. Bao, S. Cai, Y. Cao, Z. Chai, Y. Charles, H. Che, C. Chen *et al.*, "Kimi k2. 5: Visual agentic intelligence," *arXiv preprint arXiv:2602.02276*, 2026.

[34] A. H. Liu, K. Khandelwal, S. Subramanian, V. Jouault, A. Rastogi, A. Sadé, A. Jeffares, A. Jiang, A. Cahill, A. Gavaudan *et al.*, "Ministral 3," *arXiv preprint arXiv:2601.08584*, 2026.

[35] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[36] T. Glm, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Zhang, D. Rojas, G. Feng, H. Zhao *et al.*, "Chatglm: A family of large language models from glm-130b to glm-4 all tools," *arXiv preprint arXiv:2406.12793*, 2024.

[37] A. Xu, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, C. Ling *et al.*, "Deepseek-v4: Towards highly efficient million-token context intelligence," *arXiv preprint arXiv:2606.19348*, 2026.

[38] A. Chen, A. Li, B. Zhou, B. Gong, B. Jiang, B. Dan, C. Zhang, C. Yu, C. Wang, C. Ma *et al.*, "The minimax-m2 series: Mini activations unleashing max real-world intelligence," *arXiv preprint arXiv:2605.26494*, 2026.

[39] B. Xiao, B. Xia, B. Yang, B. Gao, B. Shen, C. Zhang, C. He, C. Lou, F. Luo, G. Wang *et al.*, "Mimo-v2-flash technical report," *arXiv preprint arXiv:2601.02780*,

2026.
[40] K. Krishna, Y. Song, M. Karpinska, J. Wieting, and M. Iyyer, "Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense," *Advances in neural information processing systems*, vol. 36, pp. 27 469–27 500, 2023.
[41] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, "A watermark for large language models," in *International conference on machine learning*. PMLR, 2023, pp. 17 061–17 084.
[42] A. Liu, L. Pan, X. Hu, S. Meng, and L. Wen, "A semantic invariant robust watermark for large language models," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 6499–6519.
[43] J. Huo, S. Liu, B. Wang, J. Zhang, Y. Yan, A. Liu, X. Hu, and M. Zhou, "Pmark: Towards robust and distortion-free semantic-level watermarking with channel constraints," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 27 100–27 126.
[44] J. Huo, W. Qu, Y. Yan, K. Zheng, J. Zhang, X. Hu, P. S. Yu, and M. Zhou, "Samark: A self-anchored text watermarking with paragraph-level paraphrase robustness," *arXiv preprint arXiv:2605.25796*, 2026.
[45] A. Hou, J. Zhang, T. He, Y. Wang, Y.-S. Chuang, H. Wang, L. Shen, B. Van Durme, D. Khashabi, and Y. Tsvetkov, "Semstamp: A semantic watermark with paraphrastic robustness for text generation," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024, pp. 4067–4082.
[46] K. Huang, Z. Zhang, Z. Yang, and L. Zhou, "Agent guide: A simple agent behavioral watermarking framework," *arXiv preprint arXiv:2504.05871*, 2025.
[47] H. Zhang, X. Mao, G. Dong, Z. Li, X. Su, K. Chen, J. Yang, and Z. Lin, "Memmark: State-evolution attribution watermarking for agent long-term memory systems," *arXiv preprint arXiv:2605.25002*, 2026.
[48] X. He, Q. Xu, L. Lyu, F. Wu, and C. Wang, "Protecting intellectual property of language generation apis with lexical watermark," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 10, 2022, pp. 10 758–10 766.
[49] X. He, Q. Xu, Y. Zeng, L. Lyu, F. Wu, J. Li, and R. Jia, "Cater: Intellectual property protection on text generation apis via conditional watermarks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 5431–5445, 2022.
[50] G. Yang, A. Ghasemian, F. Liu, Z. Wang, N. Mehrabi, and H. Hosseinmardi, "Asking back: Interaction-layer antidistillation watermarks," *arXiv preprint arXiv:2605.16462*, 2026.
[51] G. Cui, L. Yuan, N. Ding, G. Yao, B. He, W. Zhu, Y. Ni, G. Xie, R. Xie, Y. Lin *et al.*, "Ultrafeedback: Boosting language models with scaled ai feedback," *arXiv preprint arXiv:2310.01377*, 2023.
[52] K. Team, Y. Bai, Y. Bao, Y. Charles, C. Chen, G. Chen, H. Chen, H. Chen, J. Chen, N. Chen *et al.*, "Kimi k2: Open agentic intelligence," *arXiv preprint arXiv:2507.20534*, 2025.
[53] Anthropic, "Introducing Claude Opus 4.8," May 2026, released May 28, 2026. [Online]. Available: https://www.anthropic.com/news/claude-opus-4-8

Appendix A. Related Work

A.1. Model Distillation

Knowledge distillation is originally proposed as to transfer the capicity of a teacher model to a student model [1]. With rapid development of LLM, fine-tuning on teacher-generated data has become an increasingly important form of distillation, allowing student models to acquire advanced capabilities directly from the teacher's responses [2, 3]. This paradigm has also been extended to agent systems [4, 5, 22]. Agent Distillation [5] proposed an agent distillation method that enables student model to learn both tool-use behaviors and reasoning capabilities. AgentArk [22] introduced a method that distills multi-agent dynamics into the weights of a single model. Agent trajectories record not only generated text but also tool use and multi-step decisions, making them valuable supervision for distillation. As these trajectories can be used to cheaply transfer the capabilities of teacher agents, unauthorized distillation poses a growing threat to model copyright and calls for effective detection and tracing mechanisms.

A.2. Model Watermarking

Content watermarking. Recent studies have proposed a series of content watermarking methods to identify whether a given output is generated by a specific model. Early methods mainly embed statistical signals during token generation, such as the red-green list mechanism [41]. Later studies further improve the robustness and generation quality of watermarks by using semantic information [42–45]. However, with the development of LLM agents, traditional content watermarks are difficult to directly apply to agent settings. These methods usually embed watermark signals by changing token choices or semantic expressions in free-form text, while the key outputs of agents are often tool calls with strict format requirements. Directly modifying these outputs may break the tool calls and affect correct execution. Therefore, recent studies have started to move watermarks from generated content to agent trajectories and behavior patterns [16, 46, 47].

Model watermarking against distillation. Several watermarking methods have been proposed to protect models against unauthorized distillation. Methods such as Lexical Watermark and CATER insert special word patterns into teacher outputs, so that student models trained on these outputs can inherit detectable features [48–50]. Recent work

further targets LLM distillation by changing token generation or rewriting reasoning traces, either reducing the value of teacher outputs for distillation or leaving detectable signals in the student model [12–14]. However, these methods mainly work on text tokens or reasoning traces, and are difficult to directly apply to agents whose behavior is mainly represented by structured tool calls and multi-step interactions. Recent work has extended anti-distillation watermarking to agent trajectories [15], but its main evaluation still focuses on settings where the teacher and student models use the same base model, and its robustness to data processing such as data mixing remains limited. Therefore, a more general and robust watermarking method is still needed for tracing agent distillation.

Appendix B. Supplement of Experiments

B.1. Experimental Details

Hyperparameter selection. We select `AuxMark` hyperparameters through preliminary experiments to balance stealthiness, utility and detection reliability. For dynamic scheduling, we set $p_0 = 0.1$, $\Delta p = 0.1$, $p_{\texttt{max}} = 0.5$, and the maximum budget $B = 10$. The system generates $K = 3$ candidates, and the scoring weights in Eq. 5 are $\alpha = 0.35$, $\beta = 0.25$, and $\gamma = 0.40$. The repeatability score is $s^j_{\texttt{rep}} = 0$ for $n_j = 0$, 0.6 for $1 \le n_j < 5$, and 1 for $n_j \ge 5$. For detection, we generate $N = 3$ parameterized variants per card and use a significance level $p = 0.05$ with margin threshold $r = 0.05$. For trace-level attribution in Eq. 14, we set $\alpha' = 0.25$, $\beta' = 0.10$, and $\gamma' = 0.10$.

Baselines. We evaluate our approach against two representative agent watermarking baselines. We exclude large language model antidistillation watermarks [12, 13] from our comparison, as agent environments demand formatted outputs and modifying tokens or reasoning traces can cause format corruption and execution failures. Thus, we focus on agent schemes [16, 22, 46]. We select Seqwm [16] and Agentwm [15] because Seqwm demonstrates better robustness among trace watermarks generated for agents, and Agentwm specifically targets agent anti-distillation. Their available implementations use different watermark encodings, test units, and decision statistics from `AuxMark`; we therefore report each baseline in its native metric. Specifically, we train the student models $\mathcal{M}_s$ using the distillation dataset $\mathcal{D}_c$ watermarked by each respective scheme, and then have them replay 100 tasks from the set $\mathcal{S}$. For Agentwm, we collect unwatermarked traces using four distinct models to estimate the natural initial distribution P_c of synonymous tool sets. The scheme then modifies this P_c into a watermarked distribution $P_{\texttt{wm}}$. During detection, it extracts 5 fixed synonymous tool sets (referred to as passes) from the 100 task replays. A pass is considered successfully detected only when the JSD between the student's empirical distribution and the distribution $P_{\texttt{wm}}$ is at most 0.015. The student model is flagged as significant only if at least 3 out of 5 passes are detected ($\ge 3/5$). For Seqwm, it evaluates each replayed trace against a background null distribution generated from 1,000 incorrect keys. It uses the median p-value of these 100 test traces to determine whether the student model $\mathcal{M}_s$ is distilled, requiring a median $p < 0.05$ for a successful detection.

B.2. Supplementary Effectiveness Results

Trace-level detection. After a successful model-level detection, we further examine whether `AuxMark` can identify the specific trajectories in the suspicious set $\mathcal{S}$ that are most likely used for distillation. We separately rank the scores S_τ of the 100 trajectories from $\mathcal{S}$ on the four student models and report the arithmetic mean of `Precision`@K across the four students in Table 8. The overall average precision across the six cohorts remains at least 82.1% for every reported K. SWE-bench–GPT achieves a perfect 100.0% at P@10. These results show that `AuxMark` can identify the specific leaked data with high precision across all six cohorts.

Impact on identification accuracy. To evaluate the impact of identification accuracy on detection accuracy, we simulate different training shares by resampling our empirical results. In practice, upstream traffic monitoring is often tuned for high recall to avoid missing distillation requests, which can introduce additional benign or unrelated trajectories into the suspicious set $\mathcal{S}$. We test `AuxMark` under lower shares down to 10%. Specifically, we divide the evaluated cards for each cohort and student model into trained and untrained groups. Next, we estimate their outcome distributions and resample these two groups at various target ratios. The 50% share serves as the observed baseline. For other simulated shares, we perform 2,000 independent draws and apply the same one-sided exact sign test. As shown in Figure 5, which reports the median p-value, all 24 settings remain significant at a 20% training share. At 10%, 21 of 24 settings remain significant.

B.3. Supplementary of Robustness

Advanced data flooding. An attacker may build a diverse training set by collecting solutions to the same tasks from different teacher agents to improve the distilled model's performance [22, 51], which also dilutes the watermark signals. To evaluate the robustness of `AuxMark` against this advanced data flooding tactic, we mix the original watermarked dataset $\mathcal{D}_c$ with clean trajectories generated by alternative teacher agents. Specifically, we sequentially introduce clean data from DeepSeek-V4-Flash, MiMo-V2.5, and Qwen3-235B-A22B. In the R1, R2, and R3 settings, we incorporate data from 1, 2, and 3 alternative teachers, resulting in ratios of 1:1, 1:2, and 1:3. Then we use the combined dataset to train the student models and detect their watermarks. We evaluate a total of 24 settings across BFCL and SWE-bench, 3 dilution ratios, and 4 student models. As shown in Figure 6, `AuxMark` successfully detects every setting. Even under the most challenging R3 setting, all 8 tested student models remain successfully detected.

TABLE 6: Supplementary true-positive results for SWE-bench and Telecom. Cards is the number of cards; hit-rate cells report hits / probes (rate). w/ℓ gives card-level real / fake wins. The Sig. column indicates statistical significance: ✓ denotes $p < 0.05$, whereas × denotes $p \geq 0.05$.

Teacher Agent $\mathcal{M}_t$	Dataset	Cards	Student Model $\mathcal{M}_s$	Detection Results				
				$R^{\texttt{real}}$ (%)	$R^{\texttt{fake}}$ (%)	w/ℓ	p-value	Sig.
GPT-OSS-120B	SWE-bench	274	Mistral-24B	60 / 387 (15.5%)	18 / 387 (4.7%)	46 / 6	5.2×10^{-9}	✓
			GLM-4.7-Flash	81 / 387 (20.9%)	21 / 387 (5.4%)	67 / 6	2.0×10^{-14}	✓
			Qwen3-14B	67 / 387 (17.3%)	18 / 387 (4.7%)	54 / 4	1.6×10^{-12}	✓
			Qwen3-32B	72 / 387 (18.6%)	20 / 387 (5.2%)	58 / 6	4.5×10^{-12}	✓
	Telecom	125	Mistral-24B	83 / 146 (56.8%)	46 / 146 (31.5%)	44 / 7	6.1×10^{-8}	✓
			GLM-4.7-Flash	76 / 146 (52.1%)	42 / 146 (28.8%)	42 / 4	2.6×10^{-9}	✓
			Qwen3-14B	86 / 146 (58.9%)	44 / 146 (30.1%)	48 / 6	1.6×10^{-9}	✓
			Qwen3-32B	85 / 146 (58.2%)	36 / 146 (24.7%)	57 / 7	3.8×10^{-11}	✓
Kimi-K2.5	SWE-bench	262	Mistral-24B	85 / 437 (19.5%)	50 / 437 (11.4%)	38 / 9	1.2×10^{-5}	✓
			GLM-4.7-Flash	80 / 437 (18.3%)	42 / 437 (9.6%)	34 / 5	1.2×10^{-6}	✓
			Qwen3-14B	48 / 437 (11.0%)	17 / 437 (3.9%)	33 / 8	5.6×10^{-5}	✓
			Qwen3-32B	45 / 437 (10.3%)	14 / 437 (3.2%)	27 / 1	1.1×10^{-7}	✓
	Telecom	122	Mistral-24B	68 / 192 (35.4%)	48 / 192 (25.0%)	37 / 14	8.8×10^{-4}	✓
			GLM-4.7-Flash	56 / 192 (29.2%)	28 / 192 (14.6%)	33 / 6	7.1×10^{-6}	✓
			Qwen3-14B	60 / 192 (31.2%)	36 / 192 (18.8%)	33 / 9	1.4×10^{-4}	✓
			Qwen3-32B	66 / 192 (34.4%)	43 / 192 (22.4%)	27 / 6	1.6×10^{-4}	✓

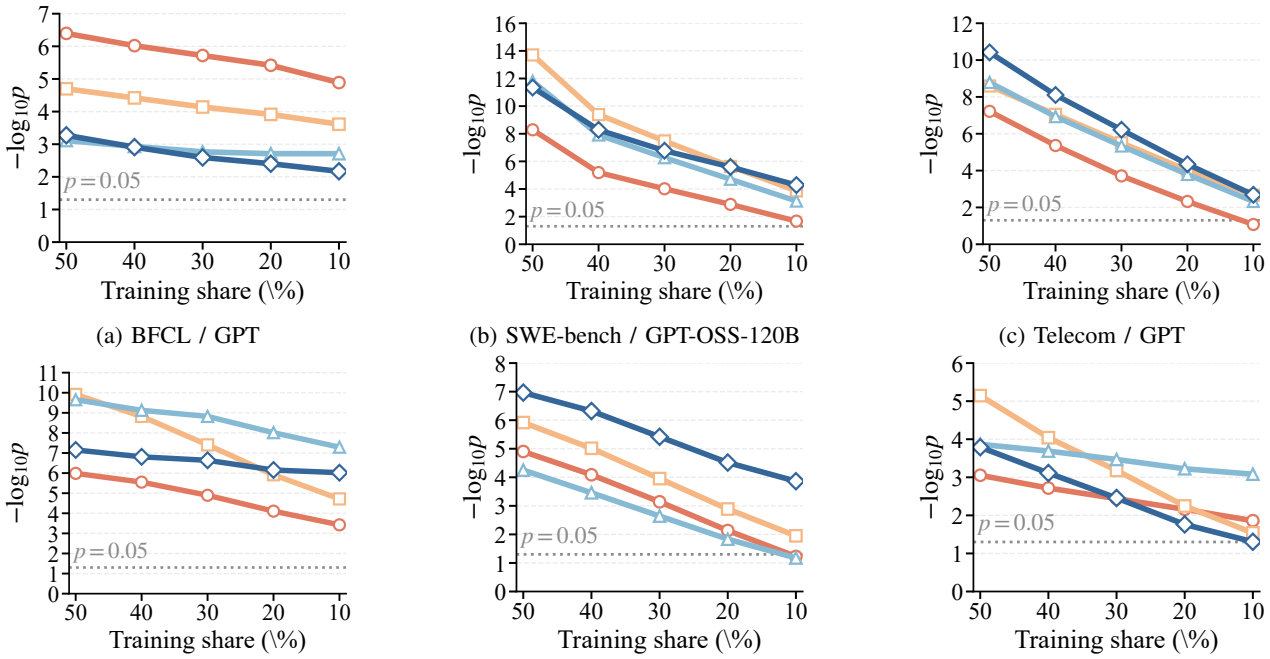
TABLE 7: Supplementary false-positive results for SWE-bench and Telecom. Cards is the number of cards; hit-rate cells report hits / probes (rate). w/ℓ gives card-level real / fake wins. The Sig. column indicates statistical significance: ✓ denotes $p < 0.05$, whereas × denotes $p \geq 0.05$.

Teacher Agent $\mathcal{M}_t$	Dataset	Cards	Clean Model $\mathcal{M}_c$	Detection Results				
				$R^{\texttt{real}}$ (%)	$R^{\texttt{fake}}$ (%)	w/ℓ	p-value	Sig.
GPT-OSS-120B	SWE-bench	274	DeepSeek-V4-Flash	0 / 387 (0.0%)	0 / 387 (0.0%)	0 / 0	1.0×10^{0}	×
			MiniMax-M2.5	5 / 387 (1.3%)	2 / 387 (0.5%)	4 / 1	1.9×10^{-1}	×
			Mistral-24B	15 / 387 (3.9%)	10 / 387 (2.6%)	10 / 6	2.3×10^{-1}	×
			Qwen3-14B	0 / 387 (0.0%)	3 / 387 (0.8%)	0 / 3	1.0×10^{0}	×
			Qwen3-32B	0 / 387 (0.0%)	2 / 387 (0.5%)	0 / 2	1.0×10^{0}	×
			Qwen3.5-Flash	4 / 387 (1.0%)	5 / 387 (1.3%)	4 / 3	5.0×10^{-1}	×
			MiMo-V2.5	3 / 387 (0.8%)	3 / 387 (0.8%)	2 / 2	6.9×10^{-1}	×
			GLM-4.7-Flash	1 / 387 (0.3%)	2 / 387 (0.5%)	1 / 2	8.8×10^{-1}	×
	Telecom	125	DeepSeek-V4-Flash	18 / 146 (12.3%)	13 / 146 (8.9%)	10 / 5	1.5×10^{-1}	×
			MiniMax-M2.5	21 / 146 (14.4%)	19 / 146 (13.0%)	6 / 3	2.5×10^{-1}	×
			Mistral-24B	11 / 146 (7.5%)	9 / 146 (6.2%)	6 / 5	5.0×10^{-1}	×
			Qwen3-14B	17 / 146 (11.6%)	21 / 146 (14.4%)	3 / 6	9.1×10^{-1}	×
			Qwen3-32B	17 / 146 (11.6%)	18 / 146 (12.3%)	2 / 3	8.1×10^{-1}	×
			Qwen3.5-Flash	11 / 146 (7.5%)	8 / 146 (5.5%)	6 / 3	2.5×10^{-1}	×
			MiMo-V2.5	13 / 146 (8.9%)	13 / 146 (8.9%)	8 / 8	6.0×10^{-1}	×
			GLM-4.7-Flash	1 / 146 (0.7%)	0 / 146 (0.0%)	1 / 0	5.0×10^{-1}	×
Kimi-K2.5	SWE-bench	262	DeepSeek-V4-Flash	12 / 437 (2.7%)	5 / 437 (1.1%)	8 / 3	1.1×10^{-1}	×
			MiniMax-M2.5	14 / 437 (3.2%)	7 / 437 (1.6%)	9 / 4	1.3×10^{-1}	×
			Mistral-24B	20 / 437 (4.6%)	11 / 437 (2.5%)	10 / 4	9.0×10^{-2}	×
			Qwen3-14B	1 / 437 (0.2%)	1 / 437 (0.2%)	1 / 1	7.5×10^{-1}	×
			Qwen3-32B	2 / 437 (0.5%)	2 / 437 (0.5%)	2 / 2	6.9×10^{-1}	×
			Qwen3.5-Flash	15 / 437 (3.4%)	13 / 437 (3.0%)	6 / 4	3.8×10^{-1}	×
			MiMo-V2.5	10 / 437 (2.3%)	6 / 437 (1.4%)	8 / 5	2.9×10^{-1}	×
			GLM-4.7-Flash	5 / 437 (1.1%)	1 / 437 (0.2%)	4 / 0	6.2×10^{-2}	×
	Telecom	122	DeepSeek-V4-Flash	11 / 192 (5.7%)	8 / 192 (4.2%)	4 / 1	1.9×10^{-1}	×
			MiniMax-M2.5	23 / 192 (12.0%)	18 / 192 (9.4%)	9 / 5	2.1×10^{-1}	×
			Mistral-24B	19 / 192 (9.9%)	16 / 192 (8.3%)	14 / 11	3.5×10^{-1}	×
			Qwen3-14B	10 / 192 (5.2%)	8 / 192 (4.2%)	7 / 5	3.9×10^{-1}	×
			Qwen3-32B	10 / 192 (5.2%)	5 / 192 (2.6%)	6 / 1	6.2×10^{-2}	×
			Qwen3.5-Flash	9 / 192 (4.7%)	3 / 192 (1.6%)	6 / 1	6.2×10^{-2}	×
			MiMo-V2.5	10 / 192 (5.2%)	7 / 192 (3.6%)	6 / 4	3.8×10^{-1}	×
			GLM-4.7-Flash	1 / 192 (0.5%)	0 / 192 (0.0%)	1 / 0	5.0×10^{-1}	×

Truncation. Before distillation, an attacker may truncate the trailing tokens of the interaction trajectories to fit the student models' context length limits or to deliberately disrupt potential watermark signals [41]. To evaluate the robustness against this operation, we remove 10%, 15%, and 20% of the tokens from the tail of each watermarked training sequence, and train the student models using these truncated trajectories. We evaluate a total of 24 settings across the BFCL and SWE-bench benchmarks, 3 truncation levels (T10, T15, and T20), and 4 student models. As shown in Figure 7, `AuxMark` successfully detects the watermark in every setting. Even under the most aggressive T20 truncation setting, all 8 tested student models remain successfully detected.

TABLE 8: Trace-level attribution accuracy. Each entry is the arithmetic mean of $\texttt{Precision}@K$ over the four student models in the cohort.

Teacher Agent $\mathcal{M}_t$	Dataset	P@10	P@15	P@20	P@25	P@30
GPT-OSS-120B	BFCL	72.5%	80.0%	80.0%	79.0%	81.7%
	SWE-bench	100.0%	98.3%	96.2%	96.0%	94.2%
	Telecom	75.0%	78.3%	80.0%	82.0%	84.2%
Kimi-K2.5	BFCL	82.5%	81.7%	82.5%	84.0%	80.8%
	SWE-bench	85.0%	86.7%	86.2%	82.0%	80.8%
	Telecom	80.0%	83.3%	82.5%	75.0%	70.8%
Overall average		**82.5%**	**84.7%**	**84.6%**	**83.0%**	**82.1%**



(a) BFCL / GPT (b) SWE-bench / GPT-OSS-120B (c) Telecom / GPT

(d) BFCL / Kimi (e) SWE-bench / Kimi-K2.5 (f) Telecom / Kimi

Figure 5: Impact of identification accuracy. The horizontal axis denotes the training share and the vertical axis denotes $-\log_{10} p$, where larger values indicate stronger evidence. In (a)–(f), the colored lines and hollow markers denote student models: Mistral-24B, GLM-4.7-Flash, Qwen3-14B, and Qwen3-32B. The ····· marks $p = 0.05$.

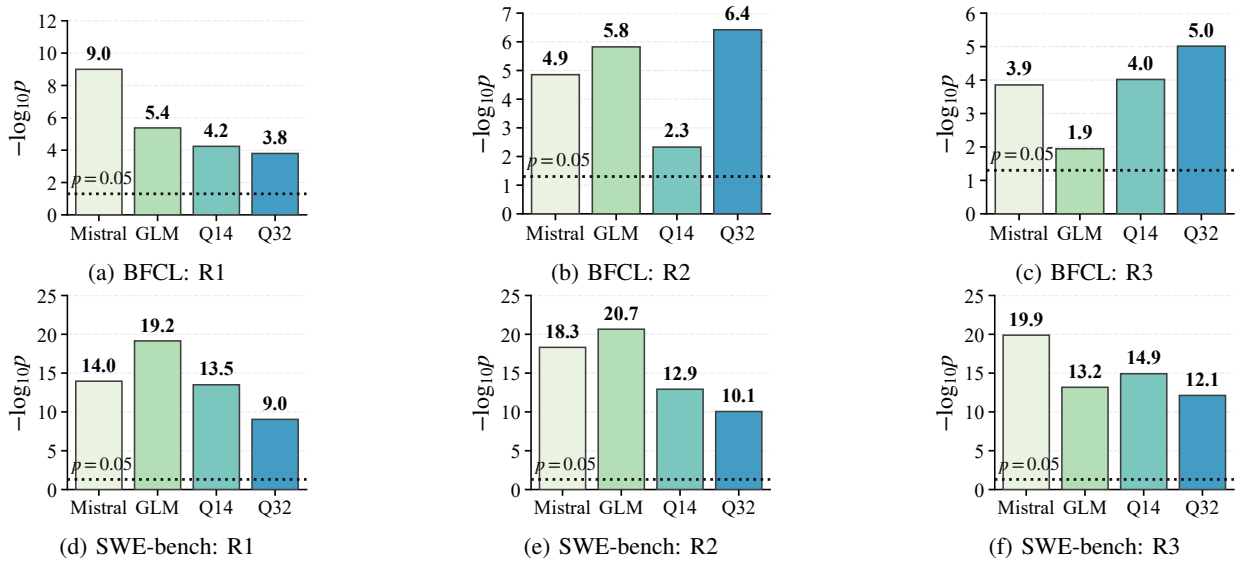
TABLE 9: Agentwm robustness to data flooding.

Dataset	Student Model $\mathcal{M}_s$	D1	D5
BFCL	Mistral-24B	$0/5$ ×	$0/5$ ×
	GLM-4.7-Flash	$0/5$ ×	$0/5$ ×
	Qwen3-14B	$2/5$ ×	$0/5$ ×
	Qwen3-32B	$1/5$ ×	$0/5$ ×
SWE-bench	Mistral-24B	$1/5$ ×	$1/5$ ×
	GLM-4.7-Flash	$2/5$ ×	$0/5$ ×
	Qwen3-14B	$0/5$ ×	$1/5$ ×
	Qwen3-32B	$0/5$ ×	$1/5$ ×

Baseline robustness. Since Seqwm does not successfully detect distilled students(Table 3), we focus our robustness evaluation on Agentwm under data flooding (the D1 and D5 settings). To ensure a rigorous comparison, we process the clean standard data to match the initial distribution P_c before mixing it with the watermarked traces. As shown in Table 9, all 16 evaluated settings fail to meet Agentwm's $\geq 3/5$ detection threshold. Agentwm encodes its watermark by shifting the tool-variant distribution towards $P_{\texttt{wm}}$. By flooding the training corpus with clean trajectories calibrated to P_c, the overall frequencies are pulled back towards the unwatermarked state. This directly dilutes the artificial distributional signal, resulting in the distilled students not sufficiently reproducing the watermarked patterns for reliable detection.

Appendix C. Discussion on Statistical Test

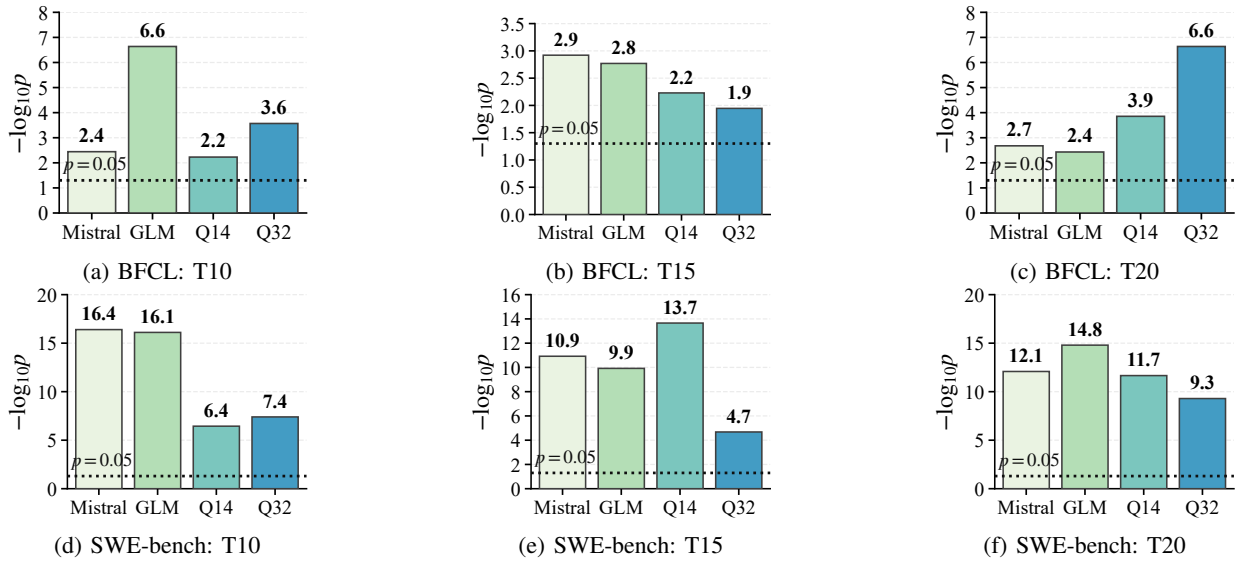
To evaluate how the choice of statistical unit affects detection results, we perform the same paired-probe tests at three granularities: card, trace, and probe levels. The card-level test aggregates parameterized probes derived from the same card, the trace-level test further aggregates all cards from the same trajectory, and the probe-level test directly treats each paired probe as an individual statistical unit. As



(a) BFCL: R1 (b) BFCL: R2 (c) BFCL: R3

(d) SWE-bench: R1 (e) SWE-bench: R2 (f) SWE-bench: R3

Figure 6: Advanced data flooding robustness. The horizontal axis lists the four student models and the vertical axis reports $-\log_{10} p$. In (a)–(f), bar colors denote Mistral-24B, GLM-4.7-Flash, Qwen3-14B, and Qwen3-32B. The ···· marks $p = 0.05$.



(a) BFCL: T10 (b) BFCL: T15 (c) BFCL: T20

(d) SWE-bench: T10 (e) SWE-bench: T15 (f) SWE-bench: T20

Figure 7: Truncation robustness. The horizontal axis lists the four student models and the vertical axis reports $-\log_{10} p$. In (a)–(f), bar colors denote Mistral-24B, GLM-4.7-Flash, Qwen3-14B, and Qwen3-32B. The ···· marks $p = 0.05$.

TABLE 10: Statistical-unit sensitivity. Entries count settings with an unadjusted one-sided sign-test $p < 0.05$; the 15% training-share row uses the median over 2,000 resamples.

Condition	Card-level	Trace-level	Probe-level
True positives	24/24	24/24	24/24
False positives	0/48	0/48	3/48
Paraphrasing attack	8/8	7/8	8/8
15% training share	24/24	22/24	24/24

shown in Table 10, the card- and trace-level tests remain consistent in the main effectiveness evaluation: both achieve 24/24 detections in the true-positive settings and 0/48 significant results in the false-positive settings. Differences mainly appear under more challenging conditions. Trace-level aggregation compresses multiple watermark releases from the same trajectory into a single sign, allowing evidence from different cards to cancel and thereby reducing statistical power and robustness. Consequently, detection decreases from 8/8 to 7/8 under the paraphrasing attack, and from 24/24 to 22/24 at a 15% training share. In contrast, although probe-level testing shows stronger apparent statistical power, it repeatedly counts parameterized probes

that share the same underlying release behavior and context, resulting in 3/48 cases with $p < 0.05$ in the false-positive settings. Although none of these cases exceeds the final fullhit-gap threshold and therefore no actual false-positive decision is triggered, the results indicate a higher false-positive risk when probes are counted individually. Based on these observations, we use the card as the primary statistical unit: it avoids repeated counting of the same watermark release event while preserving effective evidence from distinct release events.

Appendix D. Detailed Watermarking Protocols

Appendix E. Limitations of Unwatermarked Distillation Detection

Distillation Defense and Detection. Existing methods can be divided into traffic-based detection and output-side protection and detection. Traffic-based detection monitors how users query a teacher agent and examines account-level information, such as query patterns, IP addresses, and other suspicious behavior. When suspicious traffic is detected, the service provider can block, limit, or further inspect the related users and queries. Although these methods may incorrectly flag legitimate users and cannot detect every model-extraction attempt, they provide a practical first-stage screening method for our work. Based on this screening, we audit the corresponding suspicious student models to obtain direct and verifiable evidence of distillation. Another line of work modifies teacher outputs during generation or analyzes a suspicious student after training. Distillation prevention methods reduce the usefulness of teacher outputs for student training. For example, ADS adjusts the token sampling distribution, while Trace Rewriting modifies the reasoning traces produced by the teacher. Distillation detection methods can be further divided into proactive watermarking and passive fingerprinting. Proactive watermarking inserts private signals into teacher outputs. Antidistillation Fingerprinting (ADFP) embeds such signals through token sampling. For agent systems, Agentwm uses functionally equivalent tool paths to add behavioral watermarks to agent traces, while AgentMark and SeqWM add watermarks to agent planning and action sequences. Passive fingerprinting does not modify teacher outputs but compares a student with a possible teacher. PoS Templates and SCOUT analyze syntactic patterns in generated text, while Data-Free KD uses model statistics and internal representations. DistillDetect compares the likelihood scores of the student and reference models. AgentEcho analyzes similarity between tool-calling traces, and Reasoning Prefill studies how a model responds to private reasoning traces.

Systematic Limitations. Distillation prevention can make knowledge transfer more difficult, but it cannot directly identify an attacker who performs unauthorized distillation. Its effectiveness also depends strongly on assumptions about the attacker's capabilities. Fingerprinting methods do not require changes to the teacher model or service, but they usually rely on similarities between the student and teacher in generated text or execution behavior. Such similarities may also result from shared training data, common task rules, or the same agent framework. Therefore, high detection accuracy or statistically significant results alone cannot prove that distillation occurred. Some fingerprinting methods also require access to student weights, token probabilities, historical checkpoints, or a predefined set of candidate teachers. These requirements make them difficult to deploy when the student is available only through a standard black-box API. Proactive watermarking can provide more direct evidence of distillation by embedding signals controlled by a secret key or trigger. However, the reliability of this evidence depends on several conditions. The watermark must not harm model utility and must remain secret. The false-positive rate must be carefully measured and calibrated. The auditor must also rule out other ways through which the attacker could have obtained the watermarked data. Furthermore, the watermark must survive data filtering, mixed-source distillation, and subsequent training. These challenges are more serious in agent systems. Traditional token-level and text-based semantic watermarks are not designed for structured tool calls. Directly changing token sampling or rewriting tool messages may alter fields, argument values, or action semantics, causing tool calls to fail. Even if the modified JSON remains syntactically valid, the text watermark may not be preserved in the final observable tool calls. Behavioral watermarks are better suited to agent systems, but their effectiveness often decreases when the teacher and student use different base models. They are also vulnerable to watermark dilution. Some methods further introduce considerable computational cost. For example, ADFP requires a additional proxy model, while rewriting-based methods may require an auxiliary LLM. Therefore, reliable distillation detection in practical agent systems remains an open problem.

E.1. Vulnerabilities in Token-Level N-gram Matching

Panfilov et al. [19] study visible-answer overlap after prefilling a model's reasoning channel with a short teacher reasoning prefix. Their Appendix B.2 evaluates 30 Humanity's Last Exam problems using shared 1-, 2-, and 3-grams against the first 100 tokens of the reference answer, with best-of-k sampling up to $k = 100$. The visible answer is generated entirely by the target model. Their Figures 11–23 illustrate 13 public examples with problem statements, teacher reasoning excerpts, and visible-answer excerpts. Using selected public cases, we test whether alternative prefixes and shared solution patterns can produce high overlap with the same teacher reference.

We use a fixed local overlap implementation. For each candidate a and reference b, we tokenize both texts with the same regex tokenizer and retain up to their first 100 tokens.

AuxMark Watermark Embedding Protocol

Inputs: Tool schema $\mathcal{F}$, Secret key k, Trajectory ID $\texttt{id}_\tau$, Hyperparameters $(p_0, \Delta p, p_{\texttt{max}}, B, K)$.
Outputs: Watermarked trajectory τ, Private evidence card set $\mathcal{C}_\tau$.

1. Initialization Phase:

- *System Setup:*
  - Evaluate the tool schema $\mathcal{F}$ using the safety model $\mathcal{M}_{\texttt{safe}}$ to obtain the valid subset $\mathcal{F}_{\texttt{safe}}$.
  - Initialize the dynamic trigger probability $p_1 = p_0$, the remaining budget $b_1 = B$, and an empty evidence set $\mathcal{C}_\tau = \emptyset$.

2. Real-Time Embedding Phase:

For each step t immediately following a completed core action $a_{\texttt{core}}^{t-1}$, the system executes:

- *Dynamic Scheduling:*
  - Compute a cryptographic hash $u_t = \texttt{Hash}(k \parallel a \parallel \texttt{id}_\tau \parallel t) \in [0, 1)$.
  - If the budget is none ($b_t = 0$) or the condition is not met ($u_t \geq p_t$), skip and failure update.
- *Candidate Generation & Scoring:*
  - Generate K auxiliary candidates $(t_{\texttt{aux}}^j, a_{\texttt{aux}}^j)_{j=1}^K$ using the teacher model $\mathcal{M}_t$ and filter them through the validation mechanism.
  - For each valid candidate $(t_{\texttt{aux}}^j, a_{\texttt{aux}}^j)$, compute the score S_j based on naturalness ($s_{\texttt{rel}}^j$), logical consistency ($s_{\texttt{relb}}^j$), and repeatability $s_{\texttt{relb}}^j$ for the tool pairing $(f_{\texttt{core}}, f_{\texttt{aux}}^j)$.
  - Select the candidate with the highest score as $(t_{\texttt{aux}}^{\texttt{best}}, a_{\texttt{aux}}^{\texttt{best}})$. If no valid candidates exist, skip to the failure update.
- *Injection & Success Update:*
  - Inject $(t_{\texttt{aux}}^{\texttt{best}}, a_{\texttt{aux}}^{\texttt{best}})$ into the trajectory to receive the auxiliary observation $o_{\texttt{aux}}^{\texttt{best}}$.
  - Reset the probability $p_{t+1} = p_0$ and decrement the budget $b_{t+1} = b_t - 1$.
  - Increment the historical repeat count for this specific tool pairing $(f_{\texttt{core}}, f_{\texttt{aux}}^{\texttt{best}})$.
  - Create an evidence card $c_t = (h, t_{\texttt{core}}^{t-1}, a_{\texttt{core}}^{t-1}, o_{\texttt{core}}^{t-1}, f_{\texttt{aux}}^{\texttt{best}}, p_{\texttt{aux}}^{\texttt{best}})$ and append it to $\mathcal{C}_\tau$.
- *Failure/Skip Update (Executed only if no injection occurred):*
  - Increase the trigger probability $p_{t+1} = \min\{p_t + \Delta p, p_{\texttt{max}}\}$ to boost future injection chances, and retain the current budget $b_{t+1} = b_t$.

Figure 8: The watermark embedding protocol of AuxMark.

TABLE 11: Comparison of representative anti-distillation methods.

Family	Method	Acc.	Audit prior	Inject.	Verify	Evid.	Rob.	Main limitation
Distillation prevention	ADS	–	None	High	–	–	Low	Multiple measurements can bypass the defense; no post-hoc attribution.
	Trace Rewriting	–	None	High	–	–	Med.	High rewriting cost; no post-hoc attribution.
Watermarking	ADFP	BB	Same-task prompts	High	Med.	Strong	Low	Not suitable for agent settings; sensitive to watermark dilution.
	Agentwm	BB	Same-domain tasks	Low	High	Strong	Low	Weak cross-model robustness; sensitive to watermark dilution.
	AgentMark	BB	None	Med.	Low	Weak	Low	Not Survival after distillation.
	SeqWM	BB	None	Med.	Low	Weak	Med.	Not Survival after distillation.
	AuxMark	**BB**	**Suspected traces set**	**Med.**	**Med.**	**Strong**	**High**	**Requires traffic-based screening to select audit candidates.**
Fingerprinting	PoS Templates	BB/ GB	None	None	Low	Weak	Low	Closed-set attribution; weak unknown-teacher rejection.
	Data-Free KD	WB	None	None	Low	Weak	Low	Requires access to the student model's weights.
	DistillDetect	GB	Teacher and reference models	None	Low	Weak	Med.	Requires student logits and calibrated reference models.
	SCOUT	BB	None	None	Low	Weak	Med.	Confuses related models; fingerprints may be overwritten.
	AgentEcho	BB	None	None	High	Weak	Low	High AGS can be found in different graph structures.
	Reasoning Prefill	BB/ GB	None	None	Low	Weak	Low	High overlap may result from shared solutions or formatting.

BB: black-box access; GB: gray-box access; WB: white-box access. Low, Med., and High denote relative cost or robustness levels. Weak denotes indirect or non-distillation-specific evidence; Strong denotes secret-signal-based attribution.

Let $G_n(x)$ denote the set of distinct n-grams in the retained tokens of x. We compute

$$s_{\texttt{ngram}}(a, b) = \frac{\sum_{n=1}^{3} |G_n(a) \cap G_n(b)|}{\sum_{n=1}^{3} |G_n(b)|}. \tag{15}$$

Alternative prefixes and comparisons across model versions. On the geometry problem in Figure 11 of Panfilov et al. [19], Kimi-K2.6, which was publicly released before Claude Opus 4.8 [52, 53], obtains mean/best-of-4 scores of $0.274/0.565$ with the teacher opening `This is a known`, compared with $0.373/0.662$ under `This`

AuxMark Watermark Detection Protocol

Inputs: Combined evidence pool $\mathcal{C}_{\mathcal{S}}$, Suspect model $\mathcal{M}_s$, Teacher model $\mathcal{M}_t$, Hyperparameters (p, r, N).
Outputs: Detection decision $\in \{\texttt{True}, \texttt{False}\}$, and the statistical $p-\texttt{value}$.

1. Probe Construction Phase:
For each evidence card $c_i \in \mathcal{C}_{\mathcal{S}}$, the system executes:

- *Paired Probes Generation:*
  - Generate up to N_i parameterized variants using card c_i to form the real probe set $p^i_{\texttt{real}}$.
  - Construct the corresponding fake probe set $p^i_{\texttt{fake}}$ by replacing the core action in $p^i_{\texttt{real}}$ with a semantically altered fake core.

2. Evaluation Phase:
For each probe variant $j \in [1, N_i]$ of card c_i, the system executes:

- *Target Model Query & Parsing:*
  - Submit the real probe $p^i_{\texttt{real},j}$ to the suspect model $\mathcal{M}_s$ and parse the output to compute the strict fullhit indicator $h^{\texttt{real}}_{i,j} \in \{0, 1\}$.
  - Submit the fake probe $p^i_{\texttt{fake},j}$ to the suspect model $\mathcal{M}_s$ and parse the output to compute the strict fullhit indicator $h^{\texttt{fake}}_{i,j} \in \{0, 1\}$.

3. Card-Level Aggregation Phase:
For each card $c_i \in \mathcal{C}_{\mathcal{S}}$, the system executes:

- *Hit Rate & Performance Calculation:*
  - Compute the hit rates across all N_i variants: $R^{\texttt{real}}_i = \frac{1}{N_i}\sum_j h^{\texttt{real}}_{i,j}$ and $R^{\texttt{fake}}_i = \frac{1}{N_i}\sum_j h^{\texttt{fake}}_{i,j}$.
  - Calculate the performance difference: $\Delta_i = R^{\texttt{real}}_i - R^{\texttt{fake}}_i$.
  - Tally the results: increment the real win count w if $\Delta_i > 0$, and increment the fake win count ℓ if $\Delta_i < 0$ (ties where $\Delta_i = 0$ are excluded).

4. Statistical Test & Decision Phase:

- *Hypothesis Testing:*
  - Compute the one-sided exact sign-test $p-\texttt{value} = \sum_{k=w}^{w+\ell} \binom{w+\ell}{k} 2^{-(w+\ell)}$.
  - Compute the global mean difference $\Delta_{\texttt{mean}} = \frac{1}{|\mathcal{C}_{\mathcal{S}}|}\sum_i \Delta_i$.
- *Dual Criteria Check:*
  - Output `True` (Detection Successful) if both $p-\texttt{value} < p$ and $\Delta_{\texttt{mean}} \geq r$ hold. Otherwise, output `False`.

Figure 9: The watermark detection protocol of AuxMark.

is an unfamiliar and $0.250/0.415$ without a prefix. The best local score of 0.662 approaches the reported 0.80 overlap between prefilled Kimi-K3 and Opus 4.8. Within this fixed scoring setup, the alternative prefix produces a larger increase than the authentic prefix across both metrics. The selected 0.662 answer shares the opening derivation order C, F, E, D, B with the Opus 4.8 reference, as shown in Figure 10. The sampled Opus 4.6 answers use tables or different orders; the highest score among these answers is 0.343 against the same reference. Thus, this local formatting match is not exclusive to the teacher prefix or to the sampled Claude-family answers.

Convergence on standard solutions. For the linear differential equation in Figure 13 of Panfilov et al. [19], a standard solution uses a cosh integrating factor. Without a teacher prefix, GPT-OSS-120B reaches a mean overlap of 0.500 and a best-of-4 score of 0.537. Kimi-K2.6 similarly scores 0.501 without a prefix and 0.502 with the teacher prefix. The unprefilled local scores are numerically close to the reported 0.50 overlap between prefilled Kimi-K3 and Opus 4.8. The unprefilled answer excerpts are shown in Figure 11. Our results show that substantial local overlap can arise in standard solutions with conventional terminology and ordering, while the teacher prefix adds little in the Kimi-K2.6 comparison.

Response steering and sampling. Visible-answer overlap must also be interpreted in light of how responses are generated, presented, and selected. In our geometry evaluation of Kimi-K3, all 20 completions across five prefix conditions re-derive the solution, and none states the answer within its first 50 reasoning tokens, unlike the early answer in the reference trace. Higher visible overlap can therefore coexist with a different observed reasoning pattern. At the level of answer presentation, even correct solutions can receive very low scores when little matching text is available: for the problem in Figure 15 of Panfilov et al. [19], four correct answers constrained to at most 15 words each score 0.005. Such sensitivity to presentation also matters when interpreting best-of-k results, which report the largest overlap in a sampling pool. Formatting cues can therefore raise the

reported score through occasional high-overlap completions without representing typical output behavior.

E.2. Vulnerabilities in Agent Execution Graph Similarity

AgentEcho's Action Graph Similarity (AGS) averages optional-tool agreement $S_{\texttt{node}}$, sequential-pattern similarity $S_{\texttt{seq}}$, and dependency-pattern similarity $S_{\texttt{dep}}$ [17]. Moving from text to tool behavior introduces structural evidence, but its interpretation depends on which graph properties the metrics preserve.

Scale invariance and structural ambiguity. For a dependency graph G, $S_{\texttt{dep}}$ compares $\phi(G) = (r_{\texttt{reuse}}, d_{\texttt{max}}, r_{\texttt{fanout}})$ by cosine similarity [17, Appendix B.4]. These features describe the fraction of calls after the first that receive dependency inputs, the longest dependency path, and the fraction of source nodes with multiple outgoing dependency edges. Cosine similarity ignores feature-vector magnitude. Consider two graphs with 11 nodes: one contains a single dependency edge, whereas the other is a complete 10-edge chain. With path length measured in edges,

$$\phi(G_{\texttt{single}}) = (0.1, 1, 0), \qquad \phi(G_{\texttt{chain}}) = (1, 10, 0) = 10\phi(G_{\texttt{single}}). \tag{16}$$

Their $S_{\texttt{dep}}$ is exactly 1 despite large differences in reuse and dependency depth. Hence, a perfect score does not guarantee closely matching dependency structures.

Evidence from tool trajectories. We audit 75 trajectories from five models on 15 tasks and compare all ten model pairs within each task, yielding 150 comparisons. The mean $S_{\texttt{dep}}$ is 0.885, and 121 comparisons score at least 0.9. Among these, 53 have dependency edge counts differing by at least three, and 49 pair a successful trajectory with a failed one. In `retail-37`, a failed trajectory with one dependency edge and a successful trajectory with 12 edges score 0.991. We further verify candidate dependency edges with an LLM judge, using anonymized values and a prompt adapted from the published protocol [17, Appendix B.1]. Both the score and the edge-count discrepancy remain unchanged for this example. Thus, near-perfect dependency similarity persists despite substantial structural differences, even after LLM-based edge verification.

Behavioral convergence after reasoning distillation. Inspired by the reference-based formulation of Rawat et al. [18], we measure changes in behavioral similarity relative to the base checkpoint. We use the Qwen3.5-9B base model as the reference checkpoint and evaluate its publicly released reasoning-distilled variant.[2] The model card identifies Claude Opus 4.6 as the teacher. Our audit of 12,592 publicly available examples from the model card's listed datasets finds no structured tool trajectories. We use the same agent harness and user simulator for 15 τ-bench-style tasks, with one rollout per model and task. Claude Sonnet 4.5 (thinking), Kimi-K2 (thinking), and GPT-OSS-120B serve as comparison models.

2. Model card: Jackrong's reasoning-distilled Qwen3.5-9B (GGUF).

Comparison model	$\text{AGS}_{\texttt{base}}$	$\text{AGS}_{\texttt{dist}}$	$\Delta\texttt{AGS}$	$\Delta S_{\texttt{node}}$	$\Delta S_{\texttt{seq}}$	$\Delta S_{\texttt{dep}}$
Claude Sonnet 4.5 (thinking)	0.549	0.709	+0.160	-0.031	+0.425	+0.086
Kimi-K2 (thinking)	0.561	0.850	+0.289	+0.042	+0.750	+0.077
GPT-OSS-120B	0.584	0.770	+0.186	-0.047	+0.525	+0.080

TABLE 12: Changes in AGS and its components after reasoning distillation on the same 15 tasks. Component values are rounded independently.

Table 12 shows that AGS increases toward all three models, alongside a rise in task success from $4/15$ to $12/15$. The gains are dominated by $S_{\texttt{seq}}$, which increases by 0.425–0.750. In contrast, $S_{\texttt{dep}}$ increases by 0.077–0.086, while $S_{\texttt{node}}$ decreases toward both Claude and GPT-OSS. Thus, these gains primarily reflect convergence in local execution statistics and do not uniquely identify teacher-specific tool-use inheritance.

Implications for attribution. These cases expose ambiguity in interpreting high unwatermarked similarity as evidence of a specific training source. They motivate controls for shared solutions, response presentation, and general behavioral improvement, alongside calibrated false-positive rates. Our watermarking framework provides private verification signals tied to recorded teacher interactions.

Opus 4.8

```
## Setting Up Coordinates
Let me place the points on a
  number line with $A = 0$.
**Finding positions from the
  given distances:**
- $AC = 26 \Rightarrow C =
  26$
- $AF = 73 \Rightarrow F =
  73$
- $CE = 31 \Rightarrow E =
  57$
- $DF = 33 \Rightarrow D =
  73 - 33 = 40$
- $BD = 22 \Rightarrow B =
  40 - 22 = 18$
```

Kimi-K2.6 (prefilled)

```
Place the six collinear
  points on the $x$-axis.
Set $A$ at the origin, so
  $a=0$.

From the given lengths:
- $AC = 26 \Rightarrow c =
  26$
- $AF = 73 \Rightarrow f =
  73$
- $CE = 31 \Rightarrow e =
  c+31 = 57$
- $DF = 33 \Rightarrow d =
  f-33 = 40$
- $BD = 22 \Rightarrow b =
  d-22 = 18$

Thus the coordinates are
$$A(0),\; B(18),\; C(26),\;
  D(40),\; E(57),\; F(73).$$
```

Opus 4.6

```
## Setting Up Coordinates

I place the points on a
  number line with **A =
  0**. Using the given
  distances:

| Equation | Result |
|---|---|
| AF = 73 | **F = 73** |
| AC = 26 | **C = 26** |
| DF = 33 → d = 73 − 33 |
  **D = 40** |
| BD = 22 → b = 40 − 22 |
  **B = 18** |
| CE = 31 → e = 26 + 31 |
  **E = 57** |

**Verification of order:**
  A(0) < B(18) < C(26) <
  D(40) < E(57) < F(73) ✓
```

Figure 10: Excerpts from the original answers to the geometry problem in Figure 11 of Panfilov et al. [19]. Green shading marks the shared C, F, E, D, B derivation order. For each candidate model, we show the highest-overlap answer from four samples.

Opus 4.8

```
## Setting Up the Solution
The equation is linear:
$$x_3' + \tanh(t) x_3 = e^{-
  t/3}.$$
**Integrating factor:**
$$\mu(t)=\exp\left(\int\tanh
  t\,dt\right)=\exp(\ln\cosh
  t)=\cosh t.$$
```

Kimi-K2.6 (unprefilled)

```
The given first-order linear
  ODE is
$$x_3'(t)+\tanh(t)\,x_3(t)=e
  ^{-t/3}, \qquad x_3(0)=5.
  $$

**1. Solve the ODE.**
The integrating factor is
$$\mu(t)=\exp\!\left(\int
  \tanh t\,dt\right)=\exp\!
  \bigl(\ln\cosh t\bigr)=
  \cosh t.$$
```

GPT-OSS-120B (unprefilled)

```
The linear equation

\[
x_3'(t)+\tanh t\,x_3(t)=e^{-
  t/3},\qquad x_3(0)=5
\]

has integrating factor

\[
\mu(t)=\exp\!\Bigl(\int
  \tanh t\,dt\Bigr)=\exp(\ln
  \cosh t)=\cosh t .
\]
```

Figure 11: Excerpts from the original answers to the differential-equation problem in Figure 13 of Panfilov et al. [19]. Green shading marks the shared $\cosh t$ integrating factor. Each candidate has the highest reported overlap among four unprefilled samples.